Structuralia
Engineering eLearning

Enjambre cooperativo para neutralización cinética de amenazas UAS

Máster en Sistemas de Drones y Antidrones

Autor: Braulio Ernesto Prato Melo

Tutor: María Jesús Aldariz Rey



INDICE

Resumen y palabras clave 4
Introducción 5
Objetivos 7
Metodología 8

1. MARCO TEÓRICO Y ESTADO DEL ARTE 11
1.1. La amenaza UAS de baja altitud 11
1.1.1. Clasificación de los sistemas aéreos no tripulados 11
1.1.2. Tipología de la amenaza 11
1.1.3. El caso colombiano 11
1.2. Sistemas C-UAS: taxonomía y defensa por capas 12
1.2.1. Sistemas de detección en C-UAS 13
1.2.2. Sistemas de inhibición y neutralización 14
1.3. Ingeniería y comportamientos de enjambres 15
1.3.1. Leyes de Reynolds y algoritmo Boids 16
1.4. Control cooperativo líder-seguidor y consenso 17
1.5. Estado del arte en interceptación cinética dron contra dron 18
1.6. Síntesis: el enjambre cooperativo como capa cinética 20

2. MODELADO Y CONTROL COOPERATIVO LÍDER-SEGUIDOR 21
2.1. Planteamiento del problema de control 21
2.2. Escenario y supuestos de modelado 21
2.3. Modelo del dron hostil 22
2.4. Modelo del enjambre defensor 23

3. ARQUITECTURA DE INTEGRACIÓN Y VALIDACIÓN EN SIMULACIÓN 26
3.1. Arquitectura general de la co-simulación 26
3.2. Comunicación entre MATLAB y CoppeliaSim 26
3.3. Construcción de la escena 27
3.4. Ciclo de ejecución 29
3.5. Validación del comportamiento cinemático 29

4. EVALUACIÓN DE DESEMPEÑO Y ANÁLISIS DE RESULTADOS 31
4.1. Indicadores de desempeño 31
4.2. Procedimiento de evaluación 32
4.3. Resultados 32
4.3.1. Efectividad de la defensa 32
4.3.2. Tiempo de respuesta 33
4.3.3. Seguridad entre agentes propios 34
4.3.4. Análisis de sensibilidad al radio de contacto 34
4.4. Dimensionamiento del enjambre y análisis de costos 36
4.4.1. Número de defensores 36
4.4.2. Análisis de costos 37
4.5. Discusión 38

4.6. Limitaciones 40

Conclusiones 41
Líneas de trabajos futuros 42
Bibliografía 43
Anexo A. Código QuadcopterMove.m 47
Anexo B. Código Datalogger.m 52
Anexo C. Script Lua de la escena de CoppeliaSim 55

RESUMEN

La proliferación de drones de bajo costo ha creado una amenaza asimétrica en el espacio aéreo de baja altitud que las contramedidas convencionales no logran cubrir por completo. Las técnicas de tipo soft kill, como la interferencia de radiofrecuencia o la suplantación de señales GNSS, pierden eficacia frente a drones que navegan de forma autónoma o que son guiados por fibra óptica y pueden generar efectos no deseados en zonas donde las emisiones están restringidas. A esto se suma la diferencia de costos, mientras un sistema antidrón puede costar desde miles hasta millones de dólares, la amenaza que enfrenta puede no superar los dos mil dólares. En el contexto colombiano, grupos armados emplean drones comerciales estabilizados, como los de la marca DJI, cuyas cámaras facilitan el apuntamiento para lanzar granadas o artefactos explosivos improvisados. Para hacerlo, el dron debe mantenerse un momento estático sobre el objetivo, lo que lo vuelve más vulnerable a la interceptación. Este trabajo propone y valida en simulación un sistema autónomo de defensa antidrón que actúa como capa final de neutralización cinética (hard kill), basado en un enjambre cooperativo de drones con esquema líder-seguidor. El dron hostil asume el papel de líder del enjambre, de modo que los interceptores lo persiguen una vez detectado hasta que uno de ellos lo impacta.

La metodología parte de una simulación en MATLAB para evaluar el comportamiento del enjambre y del líder, y luego modifica el algoritmo para que el enjambre persiga al líder. Después se integra MATLAB, que resuelve el modelo matemático del enjambre, con CoppeliaSim, que aporta el entorno gráfico y la física mediante el motor Bullet. El escenario incluye ocho interceptores y un dron hostil en un área de 20 × 20 m, definida para reducir la carga computacional. Una interceptación se considera exitosa cuando la distancia entre los centros de los drones es menor de 0,10 m (con drones de 0,3 × 0,3 m) y se confirma por el desfase entre la posición comandada y la posición real del interceptor tras el impacto. En 40 simulaciones, el enjambre interceptó la amenaza en el 100 % de los casos con el criterio de contacto de 0,10 m y en el 90 % con un criterio más estricto, de 0,05 m.

PALABRAS CLAVE

Sistemas contra UAS (C-UAS), enjambre de drones, control líder-seguidor, inteligencia de enjambre, neutralización cinética, co-simulación.

INTRODUCCIÓN

Hace poco más de quince años, los drones eran una tecnología llamativa pero lejana para la mayoría dado el alto costo que esta demandaba para su adquisición. Hoy cualquiera puede comprar uno en internet, aprender a volarlo en una tarde y llevarlo a lugares a los que antes era imposible acceder. Esa facilidad de acceso, sumada a su bajo costo y a una autonomía cada vez mayor, ha abierto el espacio aéreo de baja altitud a casi cualquier persona. El resultado tiene dos caras. Por un lado, los sistemas aéreos no tripulados (UAS) han impulsado aplicaciones civiles valiosas en el área de rescate, agricultura y levantamiento fotográfico de alta precisión entre otras áreas de aplicación; por otro, se han convertido en una amenaza asimétrica que tanto actores estatales como no estatales han aprendido a aprovechar.

En seguridad y defensa esa asimetría se vuelve muy concreta. Un dron que cuesta unos pocos cientos de dólares puede sobrevolar una instalación estratégica para obtener información, transportar carga ilícita o lanzar un ataque directo. Lo que está en juego, en cambio, suele costar vidas, equipos e infraestructura. Ese desequilibrio entre lo poco que cuesta la amenaza y lo mucho que puede destruir es precisamente el problema que da origen a los sistemas contra aeronaves no tripuladas (C-UAS).

Colombia vive esta realidad de forma directa. Los grupos armados organizados han incorporado los drones a su forma de operar usándolos para vigilar los movimientos de la fuerza pública, para transportar estupefacientes, material de guerra y lanzar artefactos explosivos improvisados (AEI) sobre unidades militares, estaciones de policía e infraestructura crítica. La magnitud del fenómeno crece según datos del Ministerio de Defensa recogidos por el Servicio de las Naciones Unidas de Actividades Relativas a las Minas, los ataques efectivos con drones cargados de explosivos pasaron de 61 en 2024 a 333 en 2025 (UNMAS, 2026). A esto se suma un territorio de geografía difícil y montañosa, una presencia estatal dispersa en muchas regiones y un adversario cada vez más hábil en el uso de esta tecnología. Ante este escenario, las capacidades convencionales de defensa aérea se quedan cortas y se hace necesario explorar alternativas pensadas específicamente para esta amenaza.

La comunidad científica ha respondido a este reto con un campo de estudio propio. Un sistema C-UAS se entiende como aquel capaz de inhabilitar, interrumpir o tomar el control de una aeronave no tripulada de manera legal y segura (J. Wang et al., 2021) y suele organizarse en una arquitectura que combina sensores, un sistema de mitigación y un sistema de mando y control (Kang et al., 2020). En la detección se emplean radares activos y pasivos, sensores acústicos, sensores electroópticos e infrarrojos, analizadores de radiofrecuencia y tecnología LiDAR (Light Detection and Ranging), a menudo todos estos sensores integrados mediante fusión de datos; en la mitigación, las técnicas se dividen entre las de tipo soft-kill, como la suplantación de señales y la interferencia electromagnética (jamming), y las de tipo hard-kill, como los proyectiles y las armas de energía dirigida (González-Jorge et al., 2024). Las primeras tienen la ventaja de no destruir físicamente el dron, pero pierden eficacia frente a aeronaves que navegan de forma autónoma, aeronaves con enlace de fibra óptica, o aeronaves que navegan bajo visión artificial las cuales no dependen de un enlace de control basado en radiofrecuencia y su uso puede verse limitado en entornos donde las emisiones electromagnéticas están restringidas. Las segundas (hard-kill) resuelven ese problema, aunque con frecuencia a un costo elevado frente al de la amenaza que buscan neutralizar.

Una alternativa para equilibrar esa ecuación proviene de la robótica de enjambre. La idea no es nueva, hace casi cuatro décadas, Craig Reynolds (1987) mostró que el movimiento coordinado

de una bandada de aves puede reproducirse a partir de unas pocas reglas locales que cada individuo sigue por su cuenta, sin necesidad de un control central. Desde entonces ese principio se ha trasladado a sistemas de robots que mediante reglas simples e interacciones locales logran comportamientos colectivos complejos (Brambilla et al., 2013). En paralelo la teoría de control cooperativo ha dado bases formales para que un grupo de agentes llegue a acuerdos o consensos y siga una referencia común como ocurre en los esquemas líder-seguidor (Ren y Beard, 2008; Lewis et al., 2014). Los enjambres aéreos han ganado interés en aplicaciones como la vigilancia y la búsqueda, aunque buena parte de su desarrollo todavía se valida en simulación antes de llegar a entornos reales (Abdelkader et al., 2021). En Colombia, esta línea también tiene antecedentes: se han aplicado algoritmos de consenso y navegación de enjambres a la búsqueda de víctimas en zonas de desastre (León et al., 2018; Cardona y Calderón, 2019).

Esta propuesta surge además de una trayectoria concreta. El autor, ingeniero electrónico y oficial del Ejército Nacional de Colombia, se ha desempeñado entre 2024 y 2026 en el área C-UAS del Ejército, donde ha podido conocer de primera mano los límites de las soluciones basadas en radiofrecuencia frente a una amenaza que evoluciona con rapidez. El presente trabajo nace de esa experiencia: desarrollar un algoritmo de enjambre orientado a la neutralización cinética con el propósito de aumentar la probabilidad de neutralización de la amenaza.

Sobre esa base y bajo el título “Enjambre *cooperativo para neutralización cinética de amenazas UAS*”, se propone un sistema autónomo de defensa antidrón que actúa como capa final de neutralización basado en un enjambre de UAS organizado bajo un esquema líder-seguidor. Su particularidad está en invertir el papel habitual del líder: en lugar de ser un agente propio que guía al grupo, el líder es el dron hostil y el enjambre lo toma como la referencia que debe seguir. La propuesta se desarrolla y valida por completo en simulación partiendo de un modelo bidimensional y avanzando hacia una validación tridimensional mediante la co-simulación entre una plataforma de cálculo en tiempo real y un simulador de dinámica y cinemática.

OBJETIVOS

Objetivo general

Diseñar una propuesta de sistema autónomo de defensa antidrón basado en enjambres de vehículos aéreos no tripulados (UAS), bajo un esquema de control cooperativo líder-seguidor con neutralización cinética por impacto, que contribuya a fortalecer las capacidades de protección frente a amenazas aéreas de baja altitud en los escenarios de seguridad y defensa de Colombia.

Objetivos específicos

1. Modelar en un entorno de simulación bidimensional un enjambre de UAS para la defensa de una zona definida, estableciendo las bases de comportamiento y coordinación entre las plataformas.
2. Diseñar e implementar un algoritmo de control cooperativo líder-seguidor que permita al enjambre interactuar de manera autónoma con las características del terreno y adaptarse al área protegida.
3. Desarrollar una arquitectura de comunicación basada en protocolo UDP que integre la plataforma de cálculo en tiempo real (MATLAB) con el simulador de dinámica y cinemática (CoppeliaSim).
4. Validar mediante simulación en CoppeliaSim el comportamiento cinemático y la movilidad espacial de los drones.
5. Evaluar el desempeño del sistema mediante simulaciones de interceptación y neutralización de drones hostiles, determinando las rutas de intercepción y la probabilidad de derribo.

Preguntas de investigación e hipótesis

Los objetivos anteriores se concretan en cuatro preguntas de investigación, que orientan el diseño de los experimentos y a las que se responde en las conclusiones:

PI1. ¿Es suficiente una regla de cohesión dirigida al intruso, dentro de un esquema líder-seguidor basado en las reglas de Reynolds, para interceptar de forma fiable una amenaza no evasiva antes de que alcance el activo protegido?

PI2. ¿Puede el enjambre converger sobre la amenaza sin que se produzcan colisiones entre los propios defensores antes de la neutralización?

PI3. ¿Se mantiene el comportamiento del modelo bidimensional cuando los vehículos se simulan con dinámica de vuelo en CoppeliaSim, de modo que las posiciones comandadas sirvan para determinar el instante de neutralización?

PI4. ¿Cuántos defensores son necesarios para cubrir cualquier dirección de entrada de la amenaza y qué costo por neutralización tendría el sistema frente a otras alternativas?

Como hipótesis de trabajo, se plantea que un enjambre de ocho defensores, coordinado mediante una regla de cohesión dirigida al dron hostil junto con las reglas de separación y alineación de Reynolds, intercepta una amenaza no evasiva detectada con antelación en la gran mayoría de las simulaciones, sin colisiones entre defensores antes del primer contacto y con un costo por neutralización muy inferior al de los medios cinéticos convencionales.

METODOLOGÍA

Enfoque metodológico

Este trabajo sigue un enfoque aplicado y experimental basado en simulación como corresponde a un proyecto de innovación en ingeniería. Se apoya en dos componentes que se complementan: una revisión bibliográfica orientada a la temática de interés, que sustenta el estado del arte y las decisiones de diseño, y un desarrollo validado por simulación del sistema propuesto. El propósito no es formular una nueva teoría de control sino tomar un conocimiento ya consolidado (las reglas de Reynolds y el control cooperativo por consenso líder-seguidor), integrarlo y adaptarlo a un problema concreto como lo es la defensa antidrón en el contexto colombiano.

Estrategia de búsqueda bibliográfica

La búsqueda de fuentes no siguió un protocolo de revisión sistemática, sino que se orientó a la temática de interés: se identificaron las fuentes más pertinentes para el problema planteado en bases de datos científicas de alto impacto, IEEE Xplore, Scopus, Web of Science y ScienceDirect y para la producción en español, Dialnet y SciELO. Las búsquedas se realizaron durante cuatro meses de 2026. Se combinaron términos en inglés y español mediante operadores booleanos y truncamiento, y las fuentes se seleccionaron por su pertinencia para el problema y su calidad, conforme a los criterios de la Tabla 1.

Para el contexto colombiano, los datos operacionales y los costos, que rara vez se publican en revistas científicas, se recurrió de forma complementaria a literatura gris de autoría institucional verificable, informes de organismos internacionales y gubernamentales y análisis de centros de investigación en seguridad y defensa. Las notas de prensa, los contenidos divulgativos y los documentos sin revisión por pares, como los preprints, no se emplearon como fuente (Tabla 1).

Dimensión	Criterios de inclusión	Criterios de exclusión
Temática	Control cooperativo líder-seguidor y *flocking*, C-UAS cinético, detección antidrón, simulación de enjambres	Trabajos centrados solo en *soft-kill* o guerra electrónica sin componente cinético; pastoreo (*shepherding*) como método
Tipo de fuente	Artículos en revistas y congresos con revisión por pares, libros de referencia, tesis, normas técnicas y fuentes oficiales; informes de organismos internacionales, gubernamentales y centros de investigación (literatura gris), solo para contexto, datos operacionales y costos	Notas de prensa y divulgativas, preprints y contenidos sin revisión, fuentes sin autoría verificable
Ventana temporal	1987 (fundamento de Boids) y 2008–2026 para control cooperativo y C-UAS, con prioridad a 2021–2026 para la interceptación dron contra dron	Trabajos superados por revisiones posteriores del mismo grupo
Idioma	Español e inglés	Otros idiomas sin traducción verificable
Accesibilidad	Texto completo disponible y trazable	Referencias sin acceso al texto completo

Tabla 1. Criterios de inclusión y exclusión de las fuentes. Elaboración propia.

Herramientas de análisis y simulación

El desarrollo y la validación se apoyan en dos herramientas. MATLAB R2024b actúa como entorno de cálculo numérico y como cliente de la co-simulación, allí se implementa la ley de control, se generan las referencias para el enjambre, se calculan las métricas de desempeño y se hace el análisis estadístico de las simulaciones. CoppeliaSim Edu 4.10.0 actúa como servidor de dinámica, cinemática y detección de colisiones. Ambos programas se comunican mediante la Remote API de CoppeliaSim, que permite enviar en cada paso las posiciones calculadas en MATLAB a los vehículos de la escena (Capítulo 3). La síntesis del estado del arte se presenta en tablas comparativas de elaboración propia (Capítulo 1) y los resultados se ilustran con figuras generadas a partir de las trayectorias simuladas.

Diseño de los experimentos de simulación

La evaluación se realiza sobre un escenario base en el que un enjambre de ocho UAS defensores protege un área de 20 m × 20 m frente a la incursión de un dron hostil. Las dimensiones del área se definieron buscando un equilibrio entre representatividad y costo computacional, ya que la co-simulación en tiempo real entre MATLAB y CoppeliaSim aumenta de forma considerable su carga de cálculo a medida que crece el espacio simulado. Cada simulación comprende 200 pasos y el escenario se repite 40 veces con las mismas posiciones iniciales; la componente aleatoria de las velocidades iniciales hace que el comportamiento del enjambre varíe entre repeticiones, lo que permite tratar los resultados de forma estadística. Además, se analiza la sensibilidad de los resultados al radio empleado para declarar un contacto. Los resultados se analizan mediante indicadores de tiempo de respuesta, efectividad de la neutralización y seguridad entre agentes propios, cuya definición detallada se presenta en el Capítulo 4.

Registro de datos

Durante cada simulación, MATLAB registra en cada paso la posición de todos los agentes, tanto del enjambre como del dron hostil, junto con el número de iteración, que sirve de referencia temporal para todos los eventos. A partir de estas trayectorias se identifican después los contactos o posibles colisiones entre agentes, diferenciando los que ocurren con el dron hostil de los que se producen entre agentes propios. Estos registros se almacenan al final de cada simulación y constituyen la base para calcular los indicadores de desempeño y para generar las figuras de resultados.

Fases del proyecto

El desarrollo se organizó en siete fases, con realimentación entre ellas:

1. Revisión bibliográfica orientada a la temática de interés y fundamentación teórica.
2. Modelado bidimensional del líder y de los seguidores.
3. Implementación del control cooperativo líder-seguidor.
4. Integración de la lógica de detección y de la condición de neutralización.
5. Construcción de la co-simulación MATLAB–CoppeliaSim mediante la Remote API.
6. Validación funcional de los modelos.
7. Evaluación del desempeño y análisis de sensibilidad.

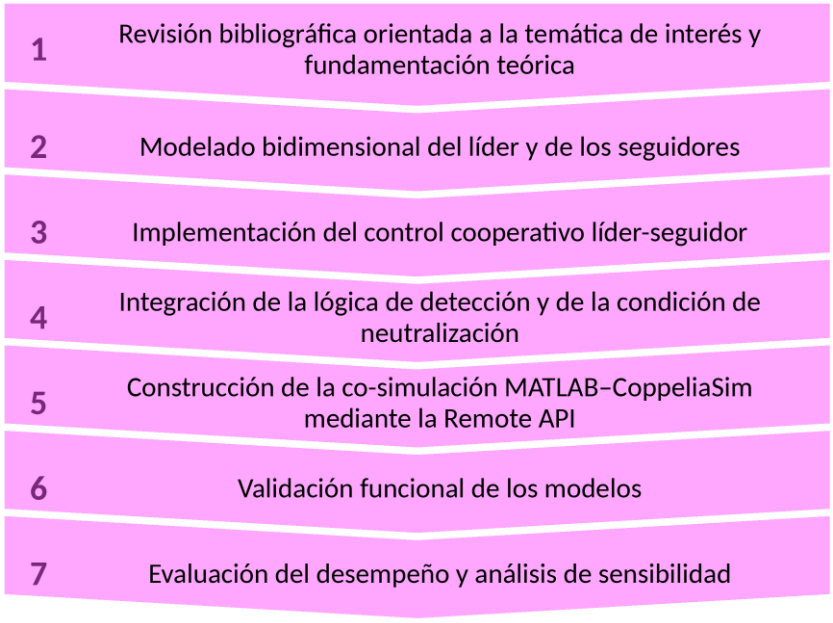


Figura 1. Proceso metodológico del proyecto

1. MARCO TEÓRICO Y ESTADO DEL ARTE

1.1. La amenaza UAS de baja altitud

1.1.1. Clasificación de los sistemas aéreos no tripulados

No existe un único criterio para clasificar los UAS. Las clasificaciones civiles y militares varían según el país y la institución y emplean atributos como el peso máximo de despegue, la altitud de operación, la velocidad o el nivel de automatización (Gettinger, 2024). En el ámbito de la defensa, una de las referencias más extendidas es la guía de clasificación de la OTAN, presentada por primera vez en 2009, que agrupa los UAS en tres clases (Gettinger, 2024). En esa guía, el peso es el criterio determinante, cuando hay discrepancia entre el peso y otro parámetro, prevalece el primero (Joint Air Power Competence Centre [JAPCC], 2010).

Clase	Peso máximo de despegue	Subcategorías	Empleo típico
Clase I	Menos de 150 kg	Micro, mini y pequeño	Unidades y subunidades tácticas
Clase II	150 a 600 kg	Táctico	Nivel de brigada
Clase III	Más de 600 kg	MALE, HALE y de ataque	Misiones de larga duración y nivel estratégico

Tabla 2. Clasificación de los UAS según la OTAN. Elaboración propia a partir de JAPCC (2010) y Gettinger (2024).

La amenaza que motiva este trabajo se concentra en la Clase I y en particular en sus subcategorías más ligeras. Por su tamaño, estos sistemas suelen ser portátiles y de lanzamiento manual y se emplean en el nivel de las unidades y subunidades tácticas (JAPCC, 2010), lo que permite desplegarlos con gran rapidez. Se trata, en su mayoría, de drones comerciales de ala rotatoria, fáciles de adquirir y de operar, que vuelan a baja altitud y presentan una firma reducida frente a los sensores convencionales de defensa aérea. En Colombia, los UAS empleados por los grupos armados son en su mayoría de Clase I, con pesos de entre 1 y 25 kg, cargas útiles de hasta 5 kg y alcances de 4 a 8 km (UNMAS, 2026); buena parte son cuadricópteros comerciales de menos de 2 kg adaptados para lanzar explosivos (Prieto-Méndez y Pirateque-Perdomo, 2025).

1.1.2. Tipología de la amenaza

Los usos hostiles de los UAS de baja altitud pueden agruparse en tres categorías. La primera es la vigilancia y el reconocimiento, que permite a un actor armado observar los movimientos de la fuerza pública, identificar vulnerabilidades y preparar acciones posteriores. La segunda es el transporte de cargas ilícitas, como estupefacientes, armas o material de guerra, aprovechando que el dron evita controles terrestres. La tercera, y la más grave, es el ataque directo, en el que el dron se emplea como plataforma para lanzar artefactos explosivos sobre un objetivo o como munición en sí mismo. Estas tres funciones no son excluyentes, un mismo actor puede emplear drones de vigilancia para orientar ataques con otros drones, lo que multiplica su efecto.

1.1.3. El caso colombiano

En Colombia, el uso de drones por grupos armados ilegales se documenta desde 2018, cuando se registró su empleo con fines de vigilancia en Tumaco, y desde 2019 se utilizan también para

lanzar explosivos (UNMAS, 2026). Las disidencias de las FARC, el ELN y el Clan del Golfo los emplean tanto con fines ofensivos como de vigilancia, en su mayoría drones comerciales adaptados (Prieto-Méndez y Pirateque-Perdomo, 2025). Desde entonces, la tendencia ha sido de crecimiento acelerado. Según el análisis de UNMAS, los ataques con drones cargados de explosivos por parte de grupos armados no estatales pasaron de 61 en 2024 a 333 en 2025, con un 63 % de los casos concentrado en Nariño, Cauca y Valle del Cauca (UNMAS, 2026). Esto supone un aumento del 445 % en un solo año y revela el volumen creciente de la amenaza que la fuerza pública debe enfrentar.

La respuesta institucional ha seguido a esta escalada. En octubre de 2025 se creó un batallón especializado en contramedidas antidrón, con unos 250 efectivos y medios de interferencia de radiofrecuencia y en diciembre de ese año el Ministerio de Defensa anunció el programa "Escudo Anti-Drones", con una inversión prevista de 1.690 millones de dólares, al que en marzo de 2026 se habían presentado 117 empresas de 27 países (UNMAS, 2026). La magnitud del fenómeno y la respuesta institucional en curso confirman que la defensa antidrón es hoy una necesidad prioritaria para la seguridad nacional de Colombia.

1.2. Sistemas C-UAS: taxonomía y defensa por capas

Un sistema contra aeronaves no tripuladas (C-UAS) se define como aquel capaz de inhabilitar, interrumpir o tomar el control de una aeronave no tripulada de forma legal y segura (J. Wang et al., 2021). Más que un dispositivo aislado, un C-UAS es una cadena de funciones, detectar la presencia de un dron, seguir su trayectoria, identificarlo como amenaza y, finalmente, mitigarlo. Kang et al. (2020) proponen una taxonomía útil para ordenar este campo, que distingue tres niveles, la plataforma, que puede ser terrestre o aérea; la arquitectura, compuesta por sistemas de sensores, de mando y control y de mitigación; y los dispositivos concretos que cumplen cada función de detección, seguimiento y neutralización.

Esta taxonomía permite entender por qué ningún sistema resuelve el problema por sí solo. La amenaza varía en altitud, alcance y tipo de plataforma y cada segmento exige soluciones distintas, con costos diferentes por neutralización (Shiffer, 2024). De ahí surge el concepto de defensa por capas, un conjunto de medios escalonados que se activan de forma progresiva a medida que la amenaza se acerca al activo protegido. Las capas exteriores suelen privilegiar la detección temprana y los medios no destructivos; las interiores, la neutralización segura de lo que logró atravesar las anteriores.

Dentro de este esquema, la capa cinética ocupa el último lugar. Es el recurso que actúa cuando los medios de interferencia no surten efecto, ya sea porque el dron navega de forma autónoma, porque el entorno no permite emitir señales o porque el tiempo de reacción es demasiado corto. Su función no es reemplazar a las capas anteriores, sino garantizar que la defensa no dependa de un único principio físico. El sistema que propone este trabajo se ubica precisamente en esa posición, como un subsistema de mitigación desplegado en plataforma aérea, que opera como capa final dentro de una arquitectura C-UAS más amplia.



Figura 2. Defensa por capas. Elaboración propia.

1.2.1. *Sistemas de detección en C-UAS*

La detección es el punto de partida de cualquier defensa antidrón, porque ningún medio de neutralización puede actuar sobre lo que no se ha visto. Las principales tecnologías empleadas son el radar activo y pasivo, los sensores acústicos, los sensores electroópticos e infrarrojos (EO/IR), los analizadores de radiofrecuencia y el LiDAR, que suelen combinarse mediante algoritmos de fusión de datos (González-Jorge et al., 2024). Cada una aporta ventajas y limitaciones distintas.

El radar es el sensor de referencia para la vigilancia de área, porque funciona de día y de noche y en condiciones meteorológicas adversas. Su principal reto frente a los drones pequeños es la baja sección radar de estos objetivos y la dificultad para distinguirlos de las aves o del ruido de fondo a baja altitud. Los sensores EO/IR, por su parte, ofrecen lo que el radar no puede: una imagen del objetivo que permite clasificarlo e identificarlo visualmente y en el caso del infrarrojo, detectarlo por su firma térmica en condiciones de baja visibilidad. Su desventaja es que requieren línea de vista y un campo de visión acotado, por lo que se emplean normalmente orientados por otro sensor. El LiDAR proporciona mediciones tridimensionales de alta precisión sobre la posición del objetivo, lo que resulta valioso para el seguimiento a corta distancia, aunque su alcance es menor y su desempeño se degrada con lluvia, niebla o polvo.

Los detectores de radiofrecuencia merecen una mención aparte, porque su eficacia depende de que el dron mantenga un enlace activo con su operador. Frente a aeronaves que vuelan rutas preprogramadas sin emitir señales, esta tecnología pierde buena parte de su utilidad (Kang et al., 2020). Esta limitación anticipa el problema que se discute en la siguiente sección.

Tecnología	Principio	Ventajas	Limitaciones
Radar	Reflexión de ondas electromagnéticas	Vigilancia de área, operación diurna y nocturna, robusto ante el clima	Baja sección radar de drones pequeños; confusión con aves y ruido de fondo
EO/IR	Imagen visible y firma térmica	Clasificación e identificación visual;	Requiere línea de vista; campo de visión

		detección térmica nocturna	acotado; sensible a niebla y lluvia
LiDAR	Pulsos láser y tiempo de vuelo	Posición 3D de alta precisión; útil para seguimiento cercano	Alcance reducido; degradación con lluvia, niebla o polvo
Radiofrecuencia	Escucha del enlace de control	Bajo costo; puede localizar al operador	Ineficaz frente a drones autónomos sin enlace activo
Acústico	Firma sonora de los motores	Pasivo y de bajo costo	Corto alcance; sensible al ruido ambiental

Tabla 3. Comparación de tecnologías de detección en sistemas C-UAS. Elaboración propia a partir de J. Wang et al. (2021), Kang et al. (2020) y González-Jorge et al. (2024).

Para los fines de este trabajo, la detección se trata como una función disponible, se asume que un sensor, sea radar, EO/IR o LiDAR, identifica la amenaza dentro de un radio de detección y entrega su posición al enjambre. El estudio de los sensores en sí queda fuera del alcance, pero su comprensión es necesaria para ubicar el sistema propuesto dentro de la cadena C-UAS completa.

1.2.2. *Sistemas de inhibición y neutralización*

Una vez detectada la amenaza, los medios de mitigación se dividen en dos grandes familias. Los de tipo soft-kill buscan interrumpir el funcionamiento del dron sin destruirlo, principalmente mediante la interferencia de sus enlaces (jamming) o la suplantación de sus señales de navegación (spoofing); los de tipo hard-kill buscan su destrucción física, mediante proyectiles o armas de energía dirigida, entre otros medios (González-Jorge et al., 2024).

Las técnicas soft-kill han sido la primera opción en muchos escenarios porque no generan restos ni daños colaterales por impacto y en algunos casos permiten recuperar el dron. Sin embargo, su eficacia depende de una condición que la amenaza puede eliminar, la existencia de un enlace que interferir. Un dron que navega de forma autónoma por una ruta preprogramada no responde a la interferencia de su enlace de control (Kang et al., 2020; Castrillo et al., 2022). A esto se suma que la emisión de señales de interferencia puede afectar las comunicaciones propias o estar restringida en ciertos entornos.

Las técnicas hard-kill, en cambio, no dependen del comportamiento electrónico del objetivo, pero enfrentan su propio problema, el costo. La proliferación de drones baratos ha creado una asimetría en la que se emplean interceptores de millones de dólares contra amenazas mucho más económicas, en Ucrania se han empleado misiles Patriot PAC-3, de más de 3 millones de dólares y AIM-9X, de cerca de 1 millón, contra drones Shahed cuyo costo se estima en unos 35.000 dólares (Hollenbeck et al., 2025). Esta relación de intercambio de costos se ha convertido en uno de los criterios centrales para evaluar la sostenibilidad de una defensa antidrón. No obstante, ese criterio no basta por sí solo, el valor de una defensa depende también de lo que protege, de modo que una interceptación costosa puede estar justificada si evita la pérdida de un activo de mayor valor (Shiffer, 2024).

Entre los medios cinéticos, los drones interceptores han surgido como una respuesta a esa asimetría. En Ucrania se ha impulsado el uso de interceptores de bajo costo, que en marzo de 2026 destruyeron más de 33.000 drones enemigos (Ministerio de Defensa de Ucrania, 2026). Castrillo et al. (2022) describen estos interceptores de colisión como vehículos desechables, a medio camino entre un dron y un misil. Este antecedente operacional es relevante porque demuestra que la neutralización de un dron mediante otro dron no es solo una idea teórica, sino una solución que ya se emplea en conflictos reales.

Mecanismo	Tipo	Ventajas	Limitaciones
Interferencia de RF (*jamming*)	Soft-kill	No destructivo; sin restos por impacto	Ineficaz frente a drones autónomos o por fibra óptica; restricciones de emisión
Suplantación de GNSS (*spoofing*)	Soft-kill	Permite desviar el dron	Solo afecta a drones que dependen de GNSS; riesgo para sistemas propios
Toma de control	Soft-kill	Permite recuperar el dron	Depende de vulnerabilidades del protocolo del dron
Proyectiles y artillería	Hard-kill	Independiente de la electrónica del objetivo	Riesgo de daño colateral; dificultad frente a blancos pequeños y ágiles
Energía dirigida	Hard-kill	Bajo costo por disparo	Alto costo de adquisición; sensible a condiciones atmosféricas
Drones interceptores	Hard-kill	Bajo costo relativo; independiente del enlace del objetivo	Se consumen en la neutralización; requieren coordinación y control precisos

Tabla 4. Comparación de mecanismos de mitigación soft-kill y hard-kill. Elaboración propia a partir de González-Jorge et al. (2024) y Kang et al. (2020).

1.3. Ingeniería y comportamientos de enjambres

La robótica de enjambre es un enfoque de la robótica colectiva que se inspira en el comportamiento autoorganizado de los animales sociales y busca que un grupo de robots relativamente simples produzca comportamientos colectivos complejos a partir de reglas sencillas e interacciones locales (Brambilla et al., 2013). Su interés para la defensa radica en tres propiedades que se derivan de esa forma de organización, la robustez, porque la pérdida de un individuo no compromete la misión del grupo; la escalabilidad, porque el sistema puede crecer o reducirse sin rediseñar su lógica; y la flexibilidad, porque el grupo se adapta a condiciones cambiantes sin necesidad de un control central.

Brambilla et al. (2013) proponen entender este campo como una ingeniería de enjambres, con métodos sistemáticos de diseño, modelado y validación. Entre los comportamientos colectivos básicos identifican la agregación, en la que los agentes se agrupan en una región; la dispersión, en la que se distribuyen para cubrir un área; y el movimiento coordinado, en el que el grupo se desplaza como una unidad cohesionada. Este último es el que más interesa a este trabajo, porque es el que permite a un enjambre seguir un objetivo móvil sin perder su organización.

En el caso particular de los enjambres aéreos, la investigación ha avanzado con rapidez, aunque todavía enfrenta retos de escalabilidad en entornos reales. La mayoría de las demostraciones se basan en simulaciones, algunas con decenas de robots en interiores controlados y pocas reportan experimentos en exteriores, debido sobre todo a las limitaciones en la localización individual y relativa de los agentes y en la tasa de intercambio de información (Abdelkader et al., 2021). Este diagnóstico justifica que la validación por simulación sea una etapa necesaria y habitual en el desarrollo de sistemas de enjambre aéreo, como la que se plantea en este trabajo.

1.3.1. Leyes de Reynolds y algoritmo Boids

El punto de partida de buena parte de los modelos de enjambre es el trabajo de Reynolds (1987), quien mostró que el movimiento coordinado de una bandada puede simularse sin programar la trayectoria de cada individuo, simplemente haciendo que cada agente, al que llamó "*boid*", siga tres reglas basadas en lo que percibe de sus vecinos cercanos:

- **Separación:** cada agente se aleja de los vecinos que están demasiado cerca, para evitar colisiones.
- **Alineación:** cada agente ajusta su velocidad para igualarla con la velocidad media de sus vecinos.
- **Cohesión:** cada agente se dirige hacia el centro de masa de sus vecinos, para mantenerse dentro del grupo.

Formalmente, la velocidad de cada agente *i* se actualiza como la suma ponderada de tres vectores, uno por cada regla:

$$v_{\mathrm{i}}(t+1) = v_{\mathrm{i}}(t) + k_s s_{\mathrm{i}} + k_a a_{\mathrm{i}} + k_c c_{\mathrm{i}}$$

donde s_i, a_i y c_i son los vectores de separación, alineación y cohesión, (k_s, k_a y k_c) son los pesos que determinan la importancia relativa de cada regla. El resultado más notable del modelo es que el comportamiento de bandada emerge de estas interacciones locales, ningún agente conoce la forma global del grupo y sin embargo el grupo se comporta como una unidad.

El modelo admite extensiones naturales. Una de ellas consiste en introducir un agente de referencia, o líder, hacia el cual se redirige la regla de cohesión, de modo que el grupo, además de mantenerse unido, siga a ese agente. Newaz et al. (2013) recogen el modelo de seguimiento de líder de Vicsek et al. (1995, como se citó en Newaz et al., 2013), en el que un agente actúa como líder del grupo mientras los demás siguen las reglas de cohesión, separación y alineación. Esta extensión es la que conecta el modelo de Reynolds con el esquema líder-seguidor que se desarrolla en las secciones siguientes.



Figura 3. Leyes de Reynolds

1.4. Control cooperativo líder-seguidor y consenso

En muchas aplicaciones no basta con que los agentes se pongan de acuerdo entre sí, se requiere que converjan hacia una referencia externa. Este es el problema del consenso líder-seguidor, también llamado seguimiento por consenso, en el que un agente líder, con estado x_0, marca la referencia y los seguidores ajustan su estado para alcanzarla. El protocolo se extiende añadiendo un término que vincula a cada seguidor con el líder:

$$\dot{x}_i = -\Sigma_j a_{ij} (x_i - x_j) - b_i (x_i - x_0)$$

donde $b_i > 0$ si el seguidor *i* recibe información directa del líder y $b_i = 0$ en caso contrario. No es necesario que todos los seguidores observen al líder, basta con que la información del líder pueda llegar a todos a través del grafo, condición que se cumple cuando el líder es la raíz de un árbol de expansión (Ren y Beard, 2008; Lewis et al., 2014).

Este esquema se ha extendido a problemas más complejos. Se han propuesto protocolos adaptativos de seguimiento de formación para enjambres de UAS con múltiples líderes y topologías de comunicación cambiantes, en los que los pesos de acoplamiento entre agentes vecinos se ajustan en el tiempo (Xie et al., 2020). Elamvazhuthi et al. (2021) estudiaron cómo un único líder puede conducir a un enjambre de seguidores hacia una distribución deseada. Y C. Wang et al. (2024) desarrollaron un modelo en el que un enjambre de robots rodea y cerca múltiples objetivos de manera uniforme, combinando redes de regulación génica con una versión mejorada del modelo de Vicsek, con aplicación explícita al cerco de objetivos con UAS en el ámbito militar.

En todos estos trabajos, el líder es un agente propio, que pertenece al sistema y cuya trayectoria se diseña o se controla. La idea que explora este trabajo consiste en invertir esa relación, el líder pasa a ser el dron hostil, un agente externo cuya trayectoria no se controla, pero que el enjambre puede observar a través de los sensores de detección. Desde el punto de vista del control cooperativo, el problema sigue siendo el de un grupo de seguidores que converge hacia la referencia que marca un líder; lo que cambia es la naturaleza de ese líder y el propósito de la convergencia.

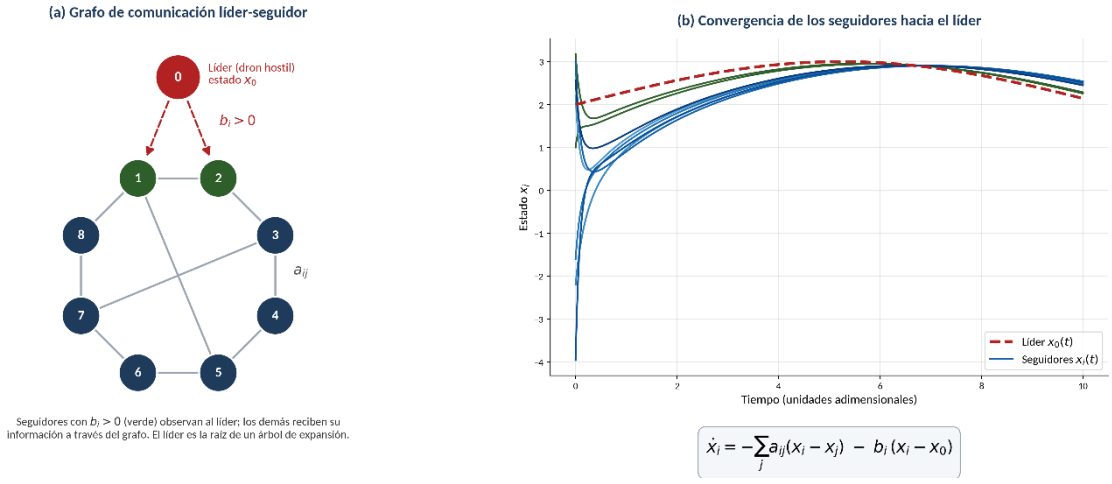


Figura 4. Esquema de consenso líder-seguidor. Elaboración propia.

1.5. Estado del arte en interceptación cinética dron contra dron

La neutralización de un dron mediante otro dron ha pasado en pocos años de ser una propuesta conceptual a constituir una línea de investigación propia dentro del campo C-UAS. En su revisión sobre equipos defensivos cooperativos de drones, Castrillo et al. (2022) distinguen entre los interceptores que capturan la amenaza, por ejemplo con redes y los que la neutralizan por colisión. Estos últimos se comportan como un híbrido entre dron y misil, porque el vehículo se consume en el impacto y plantean riesgos por la caída de restos. Los mismos autores concluyen que los equipos cooperativos de drones defensores se encontraban todavía en fase de investigación. Chamola et al. (2021), en su revisión de los ataques con UAS y de las técnicas de neutralización, sitúan los drones interceptores entre las contramedidas físicas, junto a las redes y los proyectiles.

Una primera línea de trabajo se centra en el guiado de un único interceptor. Tao et al. (2022) formularon una ley de guiado óptima con control de la velocidad terminal para interceptar cuadricópteros no cooperativos con capacidad de maniobra, validada en simulación y en vuelos reales y Pliska et al. (2024) compararon estrategias de seguimiento y captura con red para una interceptación segura en vuelo, evaluadas en un centenar de trayectorias simuladas y en vuelos reales. Estos trabajos abordan el problema terminal, es decir, cómo debe moverse un interceptor en los últimos segundos para alcanzar a un blanco que maniobra, un aspecto que queda fuera del alcance del presente trabajo.

Una segunda línea estudia la defensa con varios agentes. Brust et al. (2021) propusieron un sistema de defensa basado en un enjambre autoorganizado que intercepta al dron intruso y lo escolta fuera del área protegida sin depender de la navegación por satélite, validado en simulación. Souli et al. (2023) desarrollaron un sistema multiagente en el que varios drones perseguidores localizan a un dron no autorizado y combinan su persecución con la interferencia de sus comunicaciones, con un prototipo probado en condiciones reales. En el problema relacionado del cerco, Gao et al. (2022) diseñaron un controlador para que varios UAS rodeen a un objetivo dentro de un tubo virtual anular y C. Wang et al. (2024) propusieron un modelo de cerco uniforme de múltiples objetivos inspirado en la morfogénesis.

Los trabajos más recientes se orientan hacia la persecución cooperativa y la defensa frente a enjambres. Ren et al. (2025) emplearon aprendizaje por refuerzo multiagente, con conciencia del estado de las comunicaciones, para planificar la persecución de uno o dos objetivos por equipos de tres y cinco UAS, con tasas de éxito iguales o superiores al 90 % y una reducción del 19 % en el tiempo de captura frente a las alternativas comparadas. Xu et al. (2026) propusieron una ley de guiado cooperativo distribuida que coordina en el espacio y en el tiempo a varios interceptores, con una reducción cercana al 63 % de la energía de control en simulación. Zuo et al. (2026) plantearon un marco jerárquico de asignación de objetivos e interceptación para responder a incursiones masivas, evaluado en escenarios de hasta 50 defensores frente a 60 atacantes, con 50 ensayos por escenario.

Estos trabajos comparten varias simplificaciones con el presente estudio. La mayoría se valida solo en simulación, con modelos de masa puntual, comunicaciones ideales y atacantes que siguen trayectorias predefinidas sin reaccionar a la defensa; Zuo et al. (2026), por ejemplo, modelan a los atacantes con curvas de Bézier no evasivas y suponen comunicaciones perfectas. Tampoco existe todavía un marco común para medir el desempeño. De Cubber et al. (2025) proponen métricas estandarizadas para evaluar sistemas antidrón, entre ellas el tiempo de respuesta y la tasa de éxito de la mitigación, pero señalan que las métricas específicas de la

neutralización siguen pendientes de desarrollo. Los indicadores del Capítulo 4 se alinean con esas dos métricas.

La experiencia operacional confirma el interés de esta línea y al mismo tiempo, sus límites. Según el Ministerio de Defensa de Ucrania (2026), los drones interceptores destruyeron en marzo de 2026 más de 33.000 drones enemigos, el doble que en febrero y el programa BRAVE1 agrupa a unos cien fabricantes de interceptores. En junio de 2026, el mismo ministerio informó de interceptores que automatizan el 95 % del ciclo de interceptación de drones de tipo Shahed, desde el lanzamiento hasta la destrucción, aunque es el operador quien selecciona el objetivo y ordena el ataque (Ministerio de Defensa de Ucrania, 2026). Se trata de cifras oficiales no verificadas de forma independiente. Las tasas de interceptación del conjunto de la defensa ucraniana se sitúan entre el 90 % y el 95 % frente a los Shahed de motor de pistón, pero caen a cerca del 55 % frente a las variantes a reacción (Anokhin, 2026), lo que ilustra que la ventaja de los drones depende del contexto y de la capacidad de adaptación del adversario (Calcara et al., 2022; Kunertova, 2023).

La Tabla 5 resume estos trabajos. En conjunto, la literatura reciente aborda el guiado terminal, la asignación de objetivos y la persecución cooperativa con métodos de optimización o de aprendizaje, pero en la revisión no se identificaron trabajos que reutilicen las reglas de Reynolds con el dron hostil como líder de un esquema de consenso, ni que lo validen en una co-simulación con dinámica de vuelo. Ese es el espacio en el que se sitúa este trabajo, un mecanismo de coordinación sencillo, de bajo costo computacional y sin entrenamiento previo, evaluado en un escenario controlado como paso previo a su comparación con métodos más elaborados.

Trabajo	Enfoque	Escenario	Validación
Brust et al. (2021)	Enjambre autoorganizado que intercepta y escolta al intruso	Varios defensores frente a un intruso	Simulación
Tao et al. (2022)	Guiado óptimo con control de la velocidad terminal	Un interceptor frente a un cuadricóptero que maniobra	Simulación y vuelos reales
Souli et al. (2023)	Persecución multiagente con localización e interferencia	Varios perseguidores frente a un dron no autorizado	Simulación y prototipo real
Pliska et al. (2024)	Estrategias de seguimiento y captura con red	Un interceptor frente a un blanco en vuelo	Simulación (100 trayectorias) y vuelos reales
Ren et al. (2025)	Aprendizaje por refuerzo multiagente con conciencia de las comunicaciones	Tres contra uno y cinco contra dos	Simulación
Xu et al. (2026)	Guiado cooperativo distribuido espacio-temporal	Varios interceptores frente a un blanco	Simulación
Zuo et al. (2026)	Marco jerárquico de asignación e interceptación	Hasta 50 defensores frente a 60 atacantes no evasivos	Simulación (masa puntual)
Este trabajo	Reglas de Reynolds con el dron hostil como líder	Ocho defensores frente a una amenaza no evasiva	Co-simulación MATLAB–CoppeliaSim (40 réplicas)

Tabla 5. Trabajos recientes sobre interceptación dron contra dron y defensa C-UAS con varios agentes. Elaboración propia.

1.6. Síntesis: el enjambre cooperativo como capa cinética

La revisión anterior permite articular la propuesta de este trabajo a partir de cuatro argumentos.

El primero es la necesidad operacional. La amenaza en Colombia crece de forma sostenida y las soluciones basadas en radiofrecuencia, aunque valiosas, pierden eficacia frente a drones autónomos o controlados por medios que no se pueden interferir. Una defensa robusta requiere una capa final que no dependa del comportamiento electrónico del objetivo.

El segundo es la sostenibilidad económica. La capa cinética resuelve el problema de la dependencia del enlace, pero los interceptores tradicionales son demasiado costosos frente a amenazas baratas. Los drones interceptores han demostrado ser una forma de equilibrar esa relación y su uso ya es operacional en conflictos actuales (Ministerio de Defensa de Ucrania, 2026). La sección 4.4.2 cuantifica esta relación para el sistema propuesto.

El tercero es el aporte de la inteligencia de enjambre. Un interceptor individual depende de un solo agente para tener éxito; un enjambre, en cambio, aporta la robustez, la escalabilidad y la flexibilidad propias de los sistemas colectivos, lo que permite cubrir un área, mantener varios agentes cerca de la amenaza y compensar la pérdida de alguno de ellos.

El cuarto es la existencia de una base teórica sólida. Las reglas de Reynolds ofrecen un mecanismo simple y probado para organizar el movimiento del grupo y la teoría de consenso líder-seguidor proporciona el marco formal para analizar su convergencia hacia una referencia. La inversión del papel del líder permite reutilizar ese cuerpo de conocimiento en un problema para el que no fue concebido originalmente.

Esto no significa que el enfoque esté libre de desafíos. La validación en este trabajo se limita a la simulación, con las restricciones de escala que ello implica; la coordinación de varios agentes cerca de un objetivo introduce el riesgo de colisiones entre agentes propios; los restos de una neutralización cinética pueden generar daños en tierra y el empleo de sistemas autónomos con capacidad de neutralización plantea preguntas legales y éticas sobre el control humano de la decisión. Estos aspectos se retoman en la evaluación de resultados y en las conclusiones y delimitan con claridad el alcance de la propuesta, un esquema de control cooperativo para una capa cinética de defensa antidrón, validado en simulación como paso previo a cualquier desarrollo experimental.

2. MODELADO Y CONTROL COOPERATIVO LÍDER-SEGUIDOR

Este capítulo presenta el modelo del sistema propuesto y el algoritmo que gobierna el comportamiento del enjambre. Se parte de la formulación del problema de control, se describen el entorno de simulación y los supuestos adoptados y se desarrolla el esquema líder-seguidor que adapta las reglas de Reynolds y el consenso cooperativo al escenario de defensa antidrón. El capítulo cierra con los resultados de la simulación bidimensional, que validan el comportamiento básico del enjambre antes de su integración en el entorno tridimensional descrito en el Capítulo 3.

2.1. Planteamiento del problema de control

Se considera un enjambre de $N = 8$ vehículos aéreos no tripulados defensores, desplegados para proteger un área delimitada frente a la incursión de un dron hostil. Cada defensor se identifica con un índice $i \in \{1, \ldots, N\}$ y su estado en el instante k está dado por su posición $\mathbf{p}_i(k)$ y su velocidad $\mathbf{v}_i(k)$. El dron hostil se identifica con el índice 0 y su posición $\mathbf{p}_0(k)$ representa la referencia que el enjambre debe seguir una vez que la amenaza ha sido detectada. El número de defensores, N = 8, se justifica en la sección 4.4.1 a partir de la geometría del despliegue.

En términos de control cooperativo, el problema puede expresarse como un caso de seguimiento líder-seguidor en el que el líder no pertenece al sistema, su trayectoria no se diseña ni se controla, sino que se observa a través de los sensores de detección. El objetivo del enjambre es triple, mantener su cohesión como grupo mientras se desplaza, converger hacia la posición del líder y evitar las colisiones entre sus propios agentes. Estos tres requisitos corresponden, respectivamente, a los indicadores de tiempo de respuesta, efectividad de la neutralización y seguridad entre agentes propios definidos en la Metodología.

2.2. Escenario y supuestos de modelado

La simulación se desarrolla en un espacio bidimensional que representa, en proyección horizontal, un área protegida de 20 m × 20 m. El modelo bidimensional permite estudiar la lógica de coordinación del enjambre con un costo computacional reducido y constituye un paso previo a la validación tridimensional en CoppeliaSim.

Se adoptan los siguientes supuestos de modelado:

- **Modelo cinemático:** cada agente se representa como un punto material cuya posición se actualiza a partir de su velocidad, sin considerar la dinámica de vuelo, la cual se incorpora en la etapa de co-simulación.
- **Tiempo discreto:** la simulación avanza en pasos de tiempo fijos y cada simulación comprende 200 pasos. En la co-simulación, cada paso duró en promedio 1,0173 s en el equipo empleado (sección 3.4), por lo que las velocidades expresadas en metros por paso equivalen aproximadamente a metros por segundo.
- **Velocidad limitada:** la magnitud de la velocidad de cada agente se satura en un valor máximo de 0,1 m por paso (≈ 0,098 m/s), para representar las capacidades físicas de las plataformas.
- **Área acotada:** las posiciones de los agentes se restringen a los límites del área protegida.

- **Condiciones iniciales fijas:** las posiciones iniciales del dron hostil y de los defensores son las mismas en todas las simulaciones; la variabilidad entre simulaciones proviene de la componente aleatoria de las velocidades iniciales.
- **Percepción:** se asume que todos los defensores conocen en cada paso la posición del dron hostil, entregada por el sistema de detección, así como la posición y la velocidad de los demás defensores. No se define un radio de percepción: la vecindad es global, es decir, cada defensor interactúa con todos los demás, lo que equivale a un grafo de comunicación completo. La regla de separación, en cambio, solo actúa sobre los defensores situados a menos de 0,5 m.

2.3. Modelo del dron hostil

El dron hostil representa una amenaza que busca alcanzar un objetivo concreto dentro del área defendida. Su blanco es el activo protegido, ubicado en el punto $\mathbf{p}_a$ = (0, 0) m, que corresponde al centro del área simulada y puede interpretarse como una instalación de alta importancia, un puesto de mando, un depósito de material o una infraestructura crítica.

Al inicio de cada simulación, el dron hostil se ubica en el punto (−9,9; 9,9) m, cerca de la esquina noroeste del área. Esta condición representa un ataque que se origina fuera de la unidad y busca ingresar a la instalación. En este trabajo se evalúa un único punto de entrada; el estudio de amenazas que se aproximan desde distintas direcciones se plantea como trabajo futuro.

A partir de ese punto, el dron hostil se desplaza directamente hacia el activo protegido. En cada paso de tiempo, su posición se actualiza según:

$$p_0(k+1) = p_0(k) + v_0 \Delta t \cdot \frac{(p_a - p_0(k))}{\|p_a - p_0(k)\|} \qquad (1)$$

donde $\mathbf{p}_0(k)$ es la posición del dron hostil en el paso k, v_0 es su velocidad, considerada constante y Δt es el paso de tiempo de la simulación; en este trabajo, el avance por paso es $v_0\Delta t$ = 0,08 m, equivalente a unos 0,079 m/s. El cociente de la ecuación (x) es un vector unitario que apunta desde la posición actual del dron hacia el activo protegido, por lo que la amenaza describe una trayectoria rectilínea hacia su objetivo (Figura 5).

Este modelo corresponde a una amenaza no evasiva, es decir, que no modifica su trayectoria ante la presencia del enjambre defensor. Se trata de una simplificación deliberada por dos razones. Por un lado, permite validar el comportamiento básico del sistema en condiciones controladas, antes de enfrentarlo a escenarios más complejos. Por otro, es coherente con un tipo de amenaza especialmente relevante para este trabajo, los drones que vuelan rutas preprogramadas hacia un blanco fijo, que no dependen de un enlace con su operador y que, por esa misma razón, escapan a las contramedidas basadas en radiofrecuencia. El estudio de amenazas con trayectorias evasivas, o de ataques simultáneos con varios drones hostiles, se plantea como línea de trabajo futuro.

Finalmente, se considera que la amenaza ha alcanzado su objetivo cuando su distancia al activo protegido es menor que un radio r_a = 0,5 m, denominado radio de defensa fallida:

$$\|p_0(k) - p_a\| < r_a \qquad (2)$$

Si esta condición se cumple antes de que el enjambre neutralice la amenaza, la simulación se registra como una defensa fallida. Este criterio permite calcular, en el Capítulo 4, la proporción

de simulaciones en las que el activo resultó protegido, uno de los indicadores más directos de la efectividad del sistema.

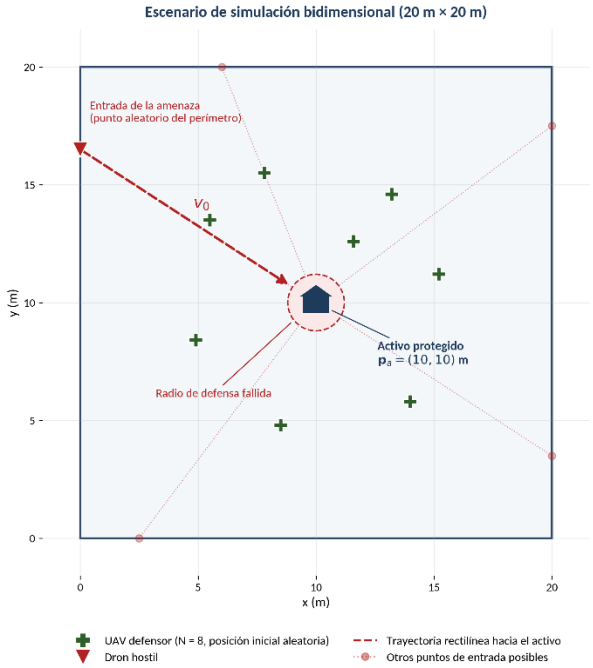


Figura 5. Escenario de simulación bidimensional

Algoritmo 1 Modelo del dron hostil

Entrada: posición inicial $\mathbf{p}_0(0) = (-9{,}9;\ 9{,}9)$ m, activo protegido $\mathbf{p}_a = (0;\ 0)$ m
Entrada: avance por paso $d_0 = 0{,}08$ m, radio de defensa fallida $r_a = 0{,}5$ m, número de pasos K
Salida: trayectoria $\{\mathbf{p}_0(k)\}$ e iteración de llegada t_a

$t_a \leftarrow \varnothing$
para $k = 0, 1, \ldots, K-1$ **hacer**
  $\mathbf{d} \leftarrow \mathbf{p}_a - \mathbf{p}_0(k)$
  si $\|\mathbf{d}\| > 0$ **entonces**
    $\mathbf{p}_0(k+1) \leftarrow \mathbf{p}_0(k) + \min(d_0, \|\mathbf{d}\|)\ \mathbf{d}/\|\mathbf{d}\|$
  si no
    $\mathbf{p}_0(k+1) \leftarrow \mathbf{p}_0(k)$
  fin si
  si $t_a = \varnothing$ **y** $\|\mathbf{p}_0(k+1) - \mathbf{p}_a\| < r_a$ **entonces**
    $t_a \leftarrow k+1$ ▷ la amenaza alcanza el radio de defensa fallida
  fin si
fin para
devolver $\{\mathbf{p}_0(k)\},\ t_a$

2.4. Modelo del enjambre defensor

El algoritmo de control propuesto se basa en una modificación deliberada de la regla de cohesión del modelo de Reynolds. En su formulación original, esta regla dirige a cada agente hacia el centro de masa del grupo, calculado como la media aritmética de las posiciones de los miembros del enjambre en el plano (*x*, *y*):

$$\overrightarrow{\boldsymbol{P(k)}} = \left(\frac{1}{N}\right)\sum_{j=1}^{N} pos_j(x_j, y_j)$$

$$\overrightarrow{\boldsymbol{cohesión}} = \left(\overrightarrow{P(k)} - pos_i(x_i, y_i)\right) * factor\ de\ cohesión$$

En el esquema propuesto, en cada iteración el centro de masa se sustituye por la posición del dron hostil, que pasa a desempeñar el papel de líder:

$$\overrightarrow{P(k)} = pos_{leader}(x, y)$$

Este cambio, aparentemente menor, transforma por completo el comportamiento colectivo. El enjambre deja de agruparse en torno a sí mismo y pasa a mantener una cohesión orientada hacia el objetivo, de modo que todos los defensores tienden a desplazarse hacia la amenaza mientras las reglas de separación y alineación conservan la organización del grupo.

Sin embargo, una cohesión dirigida exclusivamente hacia el líder resultaría demasiado agresiva y comprometería la estabilidad del enjambre. Por esta razón, la ley de actualización de la velocidad incorpora un cuarto término, asociado a la aproximación al líder, que aleja al defensor del dron hostil cuando la distancia entre ambos es menor que un umbral r_e. La velocidad de cada defensor se actualiza entonces como:

$$v(t+1) = v(t) + cohesión(x, y) + \ separación(x, y) + alineación(x, y) + avoidance(x, y)$$

En la notación de la sección 1.3.1, la cohesión, la separación y la alineación se ponderan con los pesos k_c, k_s y k_a y el término de evasión del líder (*avoidance*), con k_e. La Tabla 6 recoge los parámetros del modelo y su correspondencia con las variables del código. La alineación se calcula con la velocidad media de todo el enjambre y la separación recorre todos los demás defensores, de modo que no se emplea un radio de percepción, la vecindad es global. Con los valores empleados, el término de evasión solo actúa a menos de 0,01 m del dron hostil, por lo que no interviene antes del primer contacto con ninguno de los criterios evaluados (sección 4.1).

Símbolo	Variable en el código	Valor	Descripción
N	num_defeders	8	Número de defensores
k_c	cohesion_factor	0,02	Peso de la regla de cohesión, dirigida a la posición del dron hostil
k_s	separation_factor	0,05	Peso de la regla de separación
r_s	Valor fijo en el código	0,5 m	Radio de separación: un defensor se aleja de los que están a menos de esta distancia. Por debajo de 0,1 m se aplica un alejamiento aleatorio para evitar la división por cero
k_a	alignment_factor	0,01	Peso de la regla de alineación, calculada con la velocidad media de todo el enjambre
k_e	Enemy_avoidance_factor	0,01	Peso del término de evasión del líder (avoidance)

r_e	Enemy_separation_distance	0,01 m	Distancia al dron hostil por debajo de la cual actúa el término de evasión
r_p	—	No se define	Radio de percepción: la vecindad es global y cada defensor considera a todos los demás
$v_{máx}$	max_speed	0,1 m/paso (≈ 0,098 m/s)	Velocidad máxima de los defensores
$v_0\Delta t$	enemy_Vel	0,08 m/paso (≈ 0,079 m/s)	Avance por paso del dron hostil
$v_i(0)$	velocities	Aleatoria en [−0,5; 0,5] m/paso por componente	Velocidad inicial de cada defensor
K	num_iterations	200	Número de pasos por simulación
Δt	—	≈ 1,0173 s	Duración media de un paso en el equipo empleado (depende del equipo)

Tabla 6. Parámetros del modelo del enjambre defensor. Elaboración propia.

Algoritmo 2 Modelo de defensa cooperativa mediante enjambre de ocho UAV

Entrada: posición inicial del intruso $\mathbf{p}_e(0) = (-9{,}9;\ 9{,}9)$ m, objetivo $\mathbf{p}_o = (0;\ 0)$ m, altitud $h = 4$ m
Entrada: avance del intruso por paso $v_e = 0{,}08$ m, ocho defensores, número de iteraciones $K = 200$
Entrada: coeficientes $c_1 = 0{,}02$, $c_2 = 0{,}05$, $c_3 = 0{,}01$, $c_4 = 0{,}01$; distancia de evasión $d_e = 0{,}01$ m
Salida: trayectorias del intruso y de los defensores, posiciones reales de los UAV y registro de datos

Ubicar los defensores $\mathbf{p}_i$, $i = 1, \ldots, 8$, alrededor del objetivo
Generar velocidades iniciales aleatorias $\mathbf{v}_i \in \mathbb{R}^2$
Conectar con CoppeliaSim y obtener los identificadores de los nueve UAV y sus bases
si la conexión es correcta **entonces**
    Enviar las posiciones iniciales de los nueve objetivos de control
    para $t = 1, \ldots, K$ **hacer**
        $\mathbf{d} \leftarrow \mathbf{p}_o - \mathbf{p}_e$; $r \leftarrow \|\mathbf{d}\|$
        si $r > 10^{-9}$ **entonces**
            $\mathbf{p}_e \leftarrow \mathbf{p}_e + \min(v_e,\, r)\, \mathbf{d}/r$ ▷ avance del intruso hacia el objetivo
        fin si
        para $i = 1, \ldots, 8$ **hacer**
            $\mathbf{coh}_i \leftarrow c_1\,(\mathbf{p}_e - \mathbf{p}_i)$ ▷ cohesión hacia el intruso
            $\mathbf{s}_i \leftarrow \sum_{j \neq i} \boldsymbol{\sigma}_{ij}$, con $\boldsymbol{\sigma}_{ij}$ aleatorio si $\|\mathbf{p}_i - \mathbf{p}_j\| < 0{,}1$ y $\boldsymbol{\sigma}_{ij} = -(\mathbf{p}_j - \mathbf{p}_i)/\|\mathbf{p}_j - \mathbf{p}_i\|^2$ si $0{,}1 \leq \|\mathbf{p}_i - \mathbf{p}_j\| < 0{,}5$
            $\mathbf{sep}_i \leftarrow c_2\, \mathbf{s}_i$ ▷ separación entre defensores
            si $\|\mathbf{p}_i - \mathbf{p}_e\| < d_e$ **entonces**
                $\mathbf{eva}_i \leftarrow c_4\,(\mathbf{p}_i - \mathbf{p}_e)/\|\mathbf{p}_i - \mathbf{p}_e\|^2$ ▷ evasión del intruso
            si no
                $\mathbf{eva}_i \leftarrow \mathbf{0}$
            fin si
            $\mathbf{ali}_i \leftarrow c_3\ \text{media}(\mathbf{v}_1, \ldots, \mathbf{v}_8)$ ▷ alineación
            $\mathbf{v}_i \leftarrow \mathbf{v}_i + \mathbf{coh}_i + \mathbf{sep}_i + \mathbf{ali}_i + \mathbf{eva}_i$
            Limitar $\|\mathbf{v}_i\| \leq v_{\max} = 0{,}1$; $\mathbf{p}_i \leftarrow \mathbf{p}_i + \mathbf{v}_i$
        fin para
        Enviar a CoppeliaSim los nueve objetivos de control a la altitud h
        Leer las posiciones reales de las bases y guardarlas junto con las calculadas
        si la actualización de posiciones es correcta **entonces**
            Registrar el tiempo
        si no
            Informar el error de actualización
        fin si
    fin para
    Detener la simulación, cerrar la conexión y ejecutar `Datalogger`
fin si
devolver trayectorias $\{\mathbf{p}_e(t)\}$, $\{\mathbf{p}_i(t)\}$ y posiciones reales registradas

3. ARQUITECTURA DE INTEGRACIÓN Y VALIDACIÓN EN SIMULACIÓN

Este capítulo describe cómo se trasladó el modelo bidimensional del Capítulo 2 a un entorno de simulación tridimensional con dinámica de vuelo. Se presentan la arquitectura de comunicación entre MATLAB y CoppeliaSim, la construcción de la escena, el ciclo de ejecución de la co-simulación y el procedimiento de validación del comportamiento cinemático del enjambre.

3.1. Arquitectura general de la co-simulación

La co-simulación sigue una arquitectura cliente-servidor en la que cada herramienta cumple una función definida. MATLAB actúa como cliente, en cada paso calcula la posición del dron hostil según el modelo de la sección 2.3 y la posición deseada de cada defensor según el algoritmo de la sección 2.4, envía estas posiciones al simulador y registra los datos de la simulación. CoppeliaSim (Rohmer et al., 2013) actúa como servidor, recibe las posiciones, simula la respuesta dinámica de cada vehículo y representa el escenario en tres dimensiones (Figura 6).

Esta separación permite mantener la lógica de control en un entorno de cálculo numérico, donde es fácil de modificar y analizar, mientras el simulador aporta lo que el modelo bidimensional no incluye, la dinámica de vuelo de los cuadricópteros y su representación espacial.

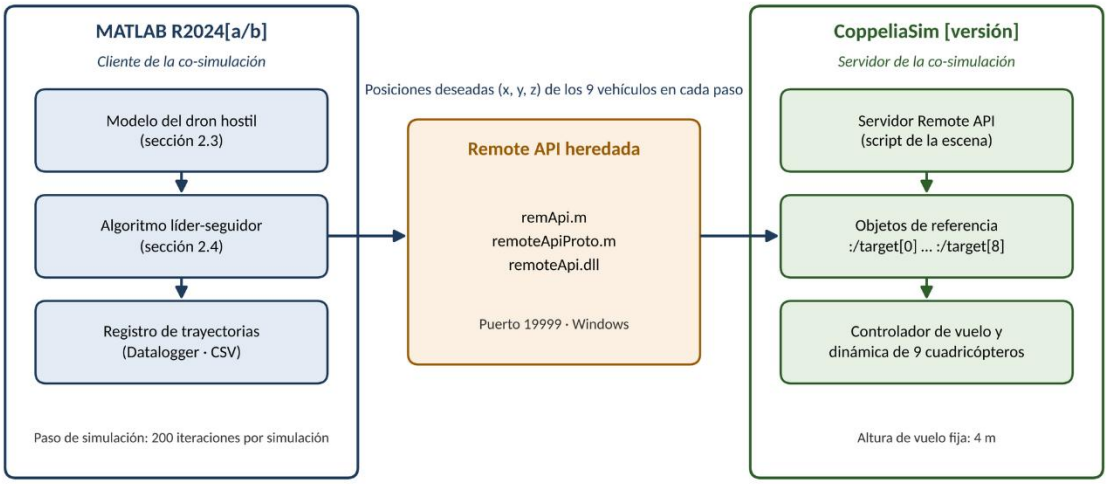


Figura 6. Arquitectura de la co-simulación entre MATLAB y CoppeliaSim. Elaboración propia.

3.2. Comunicación entre MATLAB y CoppeliaSim

La comunicación se implementó mediante la Remote API heredada de CoppeliaSim, sobre el sistema operativo Windows. Aunque inicialmente se había previsto un canal UDP desarrollado para el trabajo, se optó por esta interfaz, que comunica ambas plataformas mediante sockets, porque evitaba desarrollar un protocolo propio y ofrecía una conexión documentada. En el lado del simulador, un script asociado a un objeto de la escena inicia el servidor de la Remote API en el puerto 19999 y obtiene los identificadores de los objetos

de la escena (Anexo C). En el lado de MATLAB, la conexión se establece mediante tres archivos distribuidos con CoppeliaSim, *remoteApi.dll*, la biblioteca dinámica que implementa la comunicación del lado del cliente; *remApi.m*, la clase de MATLAB que expone las funciones de la API como métodos invocables y *remoteApiProto.m*, el archivo de prototipos que permite cargar la biblioteca sin necesidad de un compilador de C (Tabla 7).

Durante el desarrollo se identificó una incompatibilidad entre versiones del simulador, las versiones recientes de CoppeliaSim sustituyeron las funciones de acceso a los objetos de la escena por una nueva interfaz de programación. Para resolverlo, el script de la escena se actualizó a las funciones de acceso de la versión empleada, manteniendo el servidor de la Remote API heredada para la comunicación con MATLAB. La versión del simulador se fijó en CoppeliaSim Edu 4.10.0 para garantizar la reproducibilidad de los resultados.

Componente	Versión	Función
Windows	11	Sistema operativo
MATLAB	R2024b	Cálculo de la ley de control, cliente de la Remote API y registro de datos
CoppeliaSim Edu	Edu 4.10.0	Simulación de dinámica y cinemática; servidor de la Remote API; motor de física Bullet 2.78
remoteApi.dll	Distribuida con CoppeliaSim	Biblioteca de comunicación del lado del cliente
remApi.m	Distribuida con CoppeliaSim	Interfaz de MATLAB con la biblioteca
remoteApiProto.m	Distribuida con CoppeliaSim	Prototipos de funciones para cargar la biblioteca

Tabla 7. Componentes de software de la co-simulación. Elaboración propia.

3.3. Construcción de la escena

La escena de CoppeliaSim contiene nueve cuadricópteros, uno que representa al dron hostil y ocho a los defensores, además de una edificación que representa el activo protegido. Cada vehículo está compuesto por el modelo del cuadricóptero y un objeto de referencia asociado (*target*), que define la posición que el controlador de vuelo incorporado en el modelo debe alcanzar. Los objetos de referencia se identifican en la escena como ,/target[0], correspondiente al dron hostil y ,/target[1] a ,/target[8], correspondientes a los defensores 1 a 8. Todos los vehículos operan a una altura fija de 4 m, de modo que el comportamiento horizontal del enjambre corresponde al del modelo bidimensional.

Las posiciones iniciales reproducen la configuración del Capítulo 2, los ocho defensores forman un anillo alrededor del activo protegido, a 5 m de distancia sobre los ejes y las diagonales y el dron hostil inicia en (−9,9; 9,9) m, cerca de la esquina noroeste del área (Tabla 8 y Figura 7). La Figura 8 muestra la escena durante la ejecución de la co-simulación.

Agente	Objeto de referencia	Posición inicial (x; y) [m]	Altura [m]
Dron hostil	:/target[0]	(−9,9; 9,9)	4
Defensor 1	:/target[1]	(5; 0)	4
Defensor 2	:/target[2]	(5; −5)	4

Defensor 3	:/target[3]	(0; −5)	4
Defensor 4	:/target[4]	(−5; −5)	4
Defensor 5	:/target[5]	(−5; 0)	4
Defensor 6	:/target[6]	(−5; 5)	4
Defensor 7	:/target[7]	(0; 5)	4
Defensor 8	:/target[8]	(5; 5)	4
Activo protegido	—	(0; 0)	—

Tabla 8. Configuración inicial del escenario de simulación. Elaboración propia.

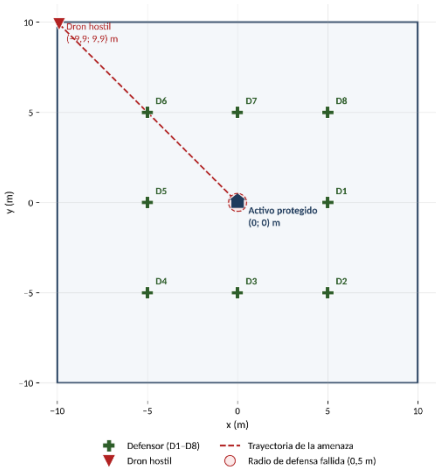


Figura 7. Configuración inicial del escenario: disposición en anillo de los defensores, punto de entrada y trayectoria del dron hostil hacia el activo protegido. Elaboración propia.

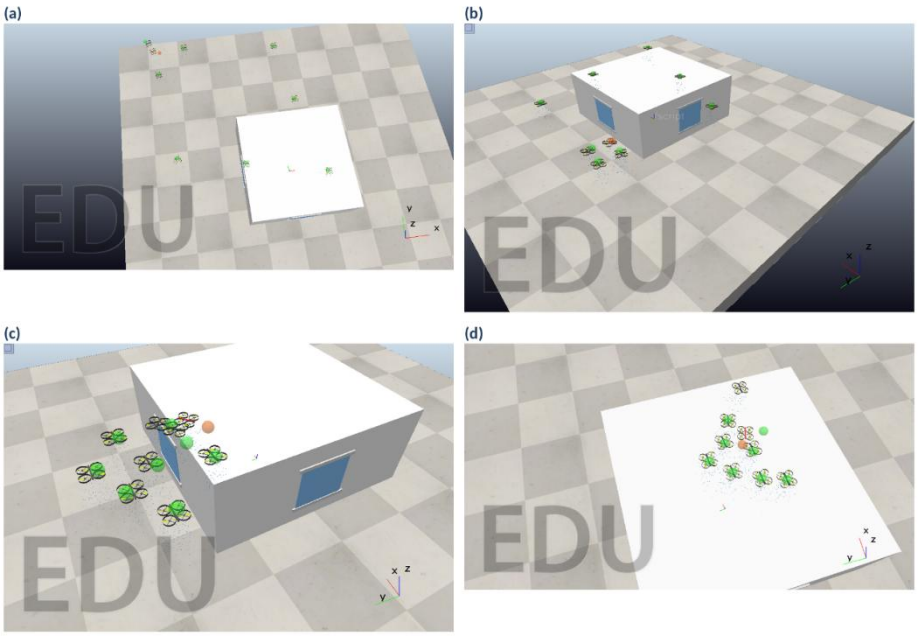


Figura 8. Escena de CoppeliaSim durante la co-simulación, (a) vista superior del área con el activo protegido y los vehículos; (b) vista en perspectiva de la edificación y el enjambre; (c) y (d) detalle del enjambre concentrado alrededor del dron hostil. Elaboración propia.

3.4. Ciclo de ejecución

Cada simulación comprende 200 pasos. En cada uno, MATLAB actualiza la posición del dron hostil según el modelo de la sección 2.3, calcula la posición deseada de cada defensor según el algoritmo de la sección 2.4 y envía las nueve posiciones a los objetos de referencia correspondientes en la escena. El controlador de vuelo de cada cuadricóptero se encarga de seguir su referencia, generando una respuesta que incluye la dinámica del vehículo. Entre pasos sucesivos se introduce una pausa de 1 s para que los vehículos respondan a la nueva referencia, de modo que cada simulación dura al menos 200s. En el equipo empleado, cada iteración duró en promedio 1,0173 s, incluida la pausa, por lo que una simulación completa duró unos 203 s. Esta duración depende del equipo y fija la escala temporal del modelo: 0,08 m por paso equivalen a unos 0,079 m/s y 0,1 m por paso, a unos 0,098 m/s.

Al finalizar la simulación, un módulo de registro (*Datalogger.m*) almacena en un archivo CSV, identificado con la fecha y hora de la simulación, la posición (x, y) del dron hostil y de los ocho defensores en cada paso. Estos archivos constituyen la base del análisis del Capítulo 4. El código de ambos módulos se presenta en los Anexos A y B, y el script Lua de la escena, en el Anexo C.

3.5. Validación del comportamiento cinemático

La validación busca comprobar que los vehículos simulados reproducen el comportamiento del modelo bidimensional cuando se incorpora la dinámica de vuelo. En las 40 simulaciones realizadas, los nueve cuadricópteros siguieron las referencias enviadas desde MATLAB durante toda la simulación, manteniendo la altura de vuelo y la disposición del enjambre previstas por el modelo, como se observa en la Figura 8.

Para cuantificarla, se realizó una simulación adicional con los mismos parámetros en la que se registraron, además de las posiciones comandadas, las posiciones reales de los nueve vehículos en el plano horizontal (x y). Se excluyó la primera iteración, en la que la lectura de las posiciones reales aún no está disponible. Antes del primer contacto, el error de seguimiento, es decir, la distancia entre la posición real y la comandada, fue en promedio de 0,10 m en los defensores y de 0,08 m en el dron hostil, con un máximo de 0,24 m. Si se compara la posición real con la referencia del paso anterior, el error medio de los defensores baja a 0,03 m, lo que indica que buena parte de él se debe a que cada vehículo sigue su referencia con un paso de retraso.

Esta simulación permitió, además, observar el primer contacto en el entorno físico (Figura 9). Entre las iteraciones 38 y 43, el defensor 6 y el dron hostil permanecieron en contacto a una distancia constante de 0,415 m entre centros, mientras sus referencias se cruzaban y ambos se separaban de ellas, el error de seguimiento del dron hostil pasó de 0,04 a 0,46 m y el del defensor, de 0,06 a 0,57 m. El choque físico comenzó en la iteración 38, el mismo instante en que se registra el primer contacto con las posiciones comandadas (iteración 38 con r_c = 0,30 m y 39 con r_c = 0,10 m), lo que respalda el uso de las posiciones comandadas para medir el instante de neutralización. Después de los choques, el error de seguimiento de los vehículos implicados aumenta, en especial a partir de la llegada de la amenaza al activo protegido, una fase que no se considera representativa (sección 4.1). Una segunda simulación de validación con los mismos parámetros mostró el mismo comportamiento: el

defensor 6 y el dron hostil permanecieron en contacto entre las iteraciones 38 y 44, a 0,418 m entre centros.

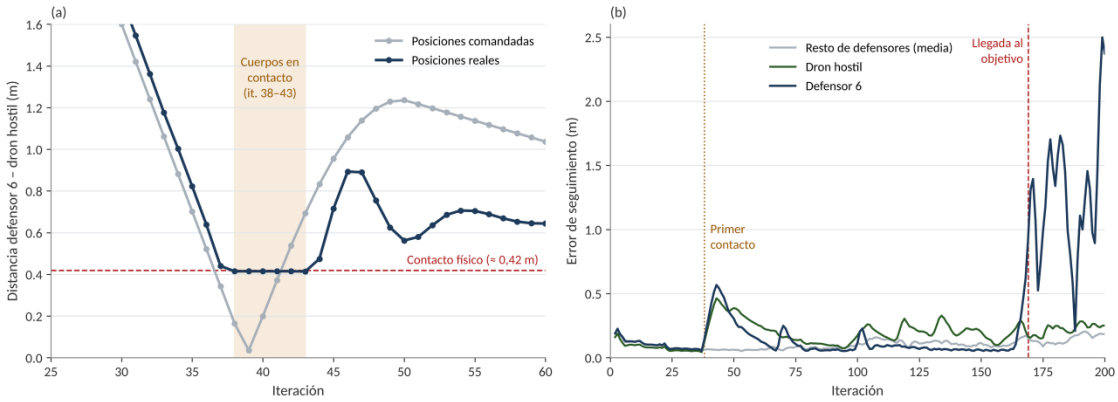


Figura 9. Simulación de validación: (a) distancia entre el defensor 6 y el dron hostil según las posiciones comandadas y reales; (b) error de seguimiento del dron hostil, del defensor 6 y media del resto de defensores. Elaboración propia.

La Figura 10 compara la trayectoria comandada y la real del dron hostil. Hasta el primer contacto, la trayectoria real coincide con la recta comandada, con una desviación lateral inferior a 0,01 m. Durante el choque, el hostil queda frenado por el defensor 6 y desplazado lateralmente; desde entonces, su trayectoria real se separa de la recta hasta 0,33 m, aunque alcanza el radio de defensa fallida en la misma iteración que la comandada (169). Este comportamiento ilustra que, en el simulador, el contacto no detiene a la amenaza, lo que confirma que lo ocurrido después del primer contacto no es representativo de una neutralización real.

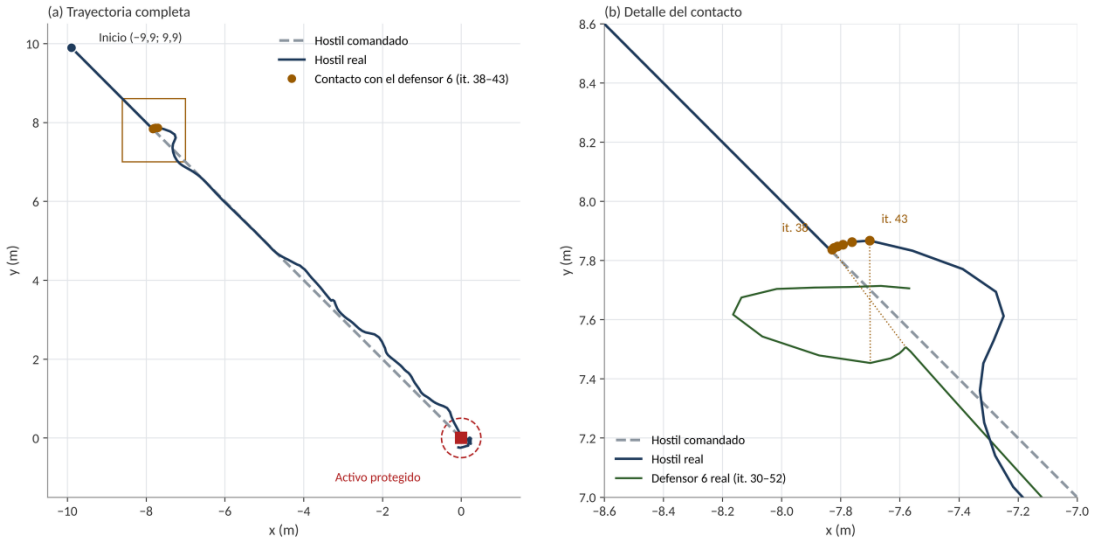


Figura 10. Trayectoria comandada y real del dron hostil en la simulación de validación: (a) trayectoria completa; (b) detalle del contacto con el defensor 6 (iteraciones 38–43). Elaboración propia.

El registro de posiciones reales se limitó al plano horizontal (x, y), en coherencia con el planteamiento bidimensional del modelo. Por ello, las distancias reales entre vehículos se refieren a su proyección horizontal y no permiten distinguir una colisión de un cruce a distinta altura; esta limitación no afecta a las métricas, que se calculan hasta el primer contacto, pero impide interpretar los acercamientos posteriores entre defensores.

4. EVALUACIÓN DE DESEMPEÑO Y ANÁLISIS DE RESULTADOS

Este capítulo evalúa el desempeño del sistema a partir de 40 ejecuciones de la co-simulación. Se definen primero los indicadores, luego se describe el procedimiento de evaluación y se presentan los resultados; después se justifica el número de defensores y se estiman los costos del sistema y finalmente, se discuten los resultados frente a la literatura.

4.1. Indicadores de desempeño

El desempeño se evalúa mediante tres grupos de indicadores, en correspondencia con lo establecido en la Metodología (Tabla 9).

Grupo	Indicador	Definición
Efectividad	Proporción de defensas exitosas	Porcentaje de simulaciones en que se registra un contacto con el dron hostil antes de que este alcance el radio de defensa fallida
Efectividad	Distancia en el primer contacto	Distancia entre el defensor que establece el primer contacto y el dron hostil en ese paso
Tiempo de respuesta	Iteración del primer contacto	Paso en que se registra el primer contacto con el dron hostil
Tiempo de respuesta	Margen de defensa	Diferencia entre la iteración en que la amenaza alcanzaría el radio de defensa fallida y la del primer contacto
Seguridad entre agentes propios	Distancia mínima entre defensores	Menor distancia entre dos defensores hasta el primer contacto
Seguridad entre agentes propios	Acercamientos entre defensores	Número de pasos, hasta el primer contacto, en que dos defensores se encuentran a menos de la distancia de contacto físico (≈ 0,42 m; sección 4.1)

Tabla 9. Indicadores de desempeño. Elaboración propia.

Se considera que existe contacto con el dron hostil cuando la distancia entre su posición y la de algún defensor es menor que el radio de contacto r = 0,10 m. El modelo de cuadricóptero empleado ocupa un área de 0,30 m × 0,30 m, hélices incluidas y todos los vehículos vuelan a la misma altura. En las simulaciones de validación descritas en la sección 3.5, un defensor y el dron hostil permanecieron en contacto a una distancia constante de 0,415–0,418 m entre centros mientras cambiaba la dirección entre ambos, lo que indica que el simulador trata cada vehículo como un cuerpo de envolvente aproximadamente circular, de unos 0,21 m de radio, equivalente a la mitad de la diagonal del vehículo. Dos vehículos reales entran, por tanto, en contacto cuando sus centros se encuentran a unos 0,42 m, con independencia de la dirección en que se aproximen. Las posiciones comandadas, en cambio, no tienen cuerpo y pueden superponerse, por lo que el criterio de contacto se aplica sobre ellas. El radio de contacto r_c = 0,10 m es, en consecuencia, mucho más exigente que la condición física de contacto, dos referencias a 0,10 m corresponden a vehículos que ya habrían chocado. Dado que la elección del radio de contacto influye en la clasificación de las simulaciones, en la sección 4.3.4 se presenta un análisis de sensibilidad con un criterio más estricto (r_c = 0,05 m). El radio de defensa fallida es r_a = 0,5 m.

En este trabajo, la neutralización se identifica con el primer contacto, la amenaza se considera neutralizada en el paso en que algún defensor establece contacto con ella y todas las métricas

de desempeño se calculan hasta ese paso. La simulación no modela el choque ni sus consecuencias, de modo que, después del primer contacto, el dron hostil continúa su trayectoria y los defensores siguen convergiendo sobre él. Ese comportamiento no es representativo de una situación real, en la que la amenaza y al menos un defensor quedarían fuera de servicio. El instante del primer contacto, en cambio, no depende de lo que ocurra después del choque, porque todos los pasos anteriores se calculan sin él. Los datos posteriores al primer contacto se muestran en algunas figuras solo como referencia.

4.2. Procedimiento de evaluación

El escenario base, descrito en las secciones 2.2, 2.3 y 3.3, se ejecutó 40 veces, con 200 pasos por simulación. Este número de réplicas permite estimar la proporción de defensas exitosas con un margen de error máximo de ±15,5 % al 95 % de confianza. Con el punto de entrada y la velocidad definidos, la amenaza alcanzaría el radio de defensa fallida en la iteración 169, por lo que todas las simulaciones concluyen con un resultado definido. Aunque las posiciones iniciales de los agentes y la trayectoria del dron hostil son idénticas en todas las simulaciones, la componente aleatoria de las velocidades iniciales y de la regla de separación hace que el comportamiento del enjambre varíe entre ellas. Por esa razón, los resultados se reportan como proporciones con su intervalo de confianza de Wilson (1927) y como valores medios con su desviación estándar (DE) y no a partir de una simulación individual. Los contactos se calcularon a partir de las posiciones registradas en los archivos CSV de cada simulación.

4.3. Resultados

4.3.1. Efectividad de la defensa

Con el radio de contacto r_c = 0,10 m, el enjambre registró contacto con el dron hostil antes de que este alcanzara el radio de defensa fallida en las 40 simulaciones, lo que corresponde a una proporción de defensas exitosas del 100 % (IC 95 % de Wilson: 91,2 % – 100 %). En el paso del primer contacto, la distancia entre el defensor y el dron hostil fue en promedio de 0,065 m (DE 0,026 m), con valores entre 0,008 y 0,097 m (Tabla 10). Como el radio de contacto de 0,10 m es más exigente que la condición física de colisión (sección 4.1), esta proporción de defensas exitosas es una cota conservadora, toda simulación clasificada como exitosa implica una colisión física con la amenaza. La Figura 11 muestra las trayectorias de todos los agentes en una simulación representativa.

Indicador	Resultado
Defensas exitosas	40/40 (100 %; IC 95 %: 91,2 % – 100 %)
Iteración del primer contacto	Media 38,8 (DE 0,7); rango 38 – 40
Margen de defensa	Media 130,2 iteraciones (DE 0,7)
Distancia en el primer contacto	Media 0,065 m (DE 0,026); rango 0,008 – 0,097 m
Distancia mínima entre defensores hasta el primer contacto	Media 3,91 m (DE 0,07); rango 3,77 – 4,02 m
Simulaciones con colisión entre defensores hasta el primer contacto	0/40

Tabla 10. Resultados de los indicadores de desempeño sobre 40 simulaciones (r_c = 0,10 m). Elaboración propia.

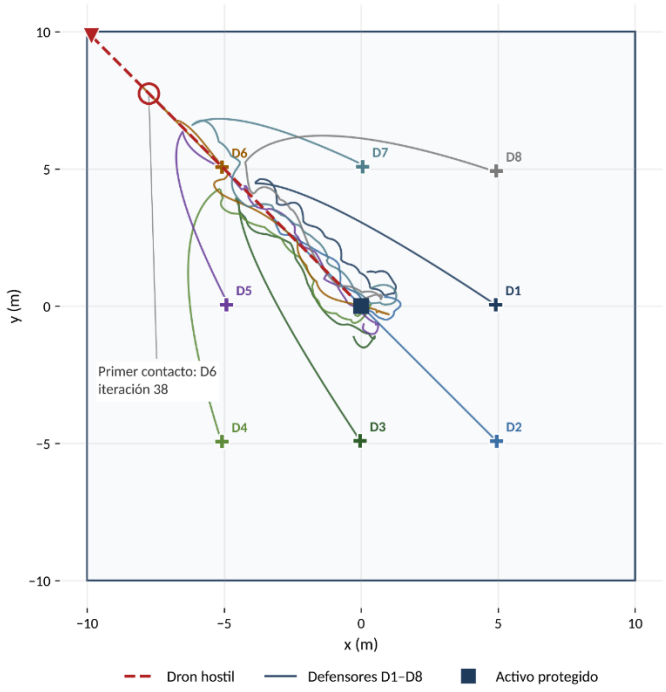


Figura 11. Trayectorias del dron hostil y de los ocho defensores en una simulación representativa, con la posición del primer contacto. Las trayectorias posteriores al primer contacto se muestran solo como referencia. Elaboración propia.

4.3.2. Tiempo de respuesta

El primer contacto se produjo en promedio en la iteración 38,8 (DE 0,7), mientras que, en ausencia de contacto, la amenaza habría alcanzado el radio de defensa fallida en la iteración 169. El sistema dispuso así de un margen de defensa medio de 130 iteraciones, equivalente a unos 132 s con la duración media de paso del equipo empleado. La Figura 12 muestra la evolución de la distancia de cada defensor al dron hostil en la misma simulación representativa.

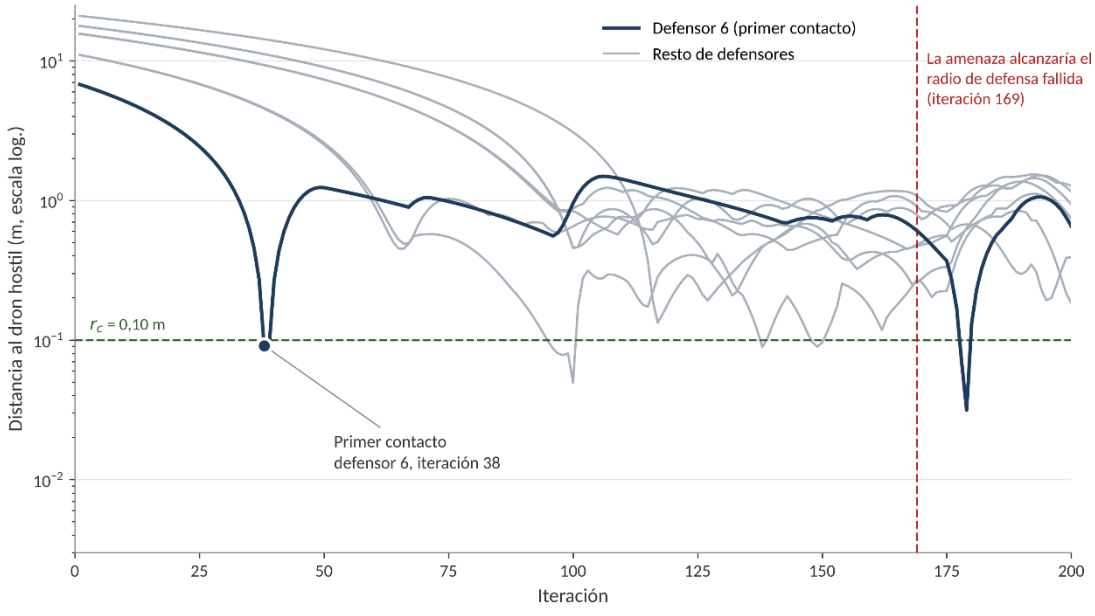


Figura 12. Distancia de cada defensor al dron hostil en una simulación representativa, con el

radio de contacto y la iteración en que la amenaza alcanzaría el radio de defensa fallida. La evolución posterior al primer contacto se muestra solo como referencia. Elaboración propia.

4.3.3. Seguridad entre agentes propios

Hasta el primer contacto, la menor distancia entre dos defensores fue en promedio de 3,91 m (DE 0,07 m), con un mínimo de 3,77 m, muy por encima de los 0,42 m a los que dos vehículos entran en contacto (Figura 13). En ninguna de las 40 simulaciones hubo, por tanto, colisiones entre defensores antes de la neutralización. Esto se explica porque el primer contacto se produce cuando la amenaza alcanza al defensor situado sobre su trayectoria, mientras el resto del enjambre todavía se encuentra en fase de aproximación. Después del primer contacto, los defensores siguen convergiendo sobre una amenaza que en la simulación no se detiene y la distancia mínima entre sus referencias baja a 0,37 m en promedio, por debajo de la distancia de contacto en todas las simulaciones. Estos acercamientos no son representativos de una situación real, porque se producen después del choque, pero indican que el radio de la regla de separación (0,50 m) no garantiza por sí solo una distancia libre de colisión cuando el enjambre se concentra sobre un mismo punto.

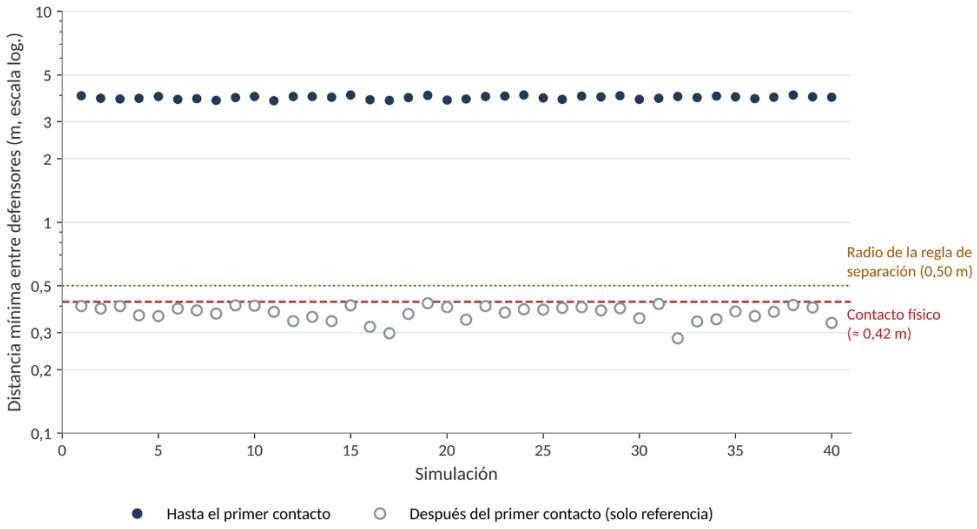


Figura 13. Distancia mínima entre defensores en cada simulación, antes y después del primer contacto (r = 0,10 m), frente a la distancia de contacto físico y al radio de la regla de separación. Los valores posteriores al primer contacto se muestran solo como referencia. Elaboración propia.

4.3.4. Análisis de sensibilidad al radio de contacto

Para evaluar la dependencia de los resultados respecto del criterio de contacto, se repitió el análisis con un radio más estricto, r_c = 0,05 m (Tabla 11). Con este criterio, la proporción de defensas exitosas fue del 90 % (36 de 40; IC 95 %: 76,9 % – 96,0 %). De las cuatro simulaciones no exitosas, en dos el primer contacto se produjo después de que la amenaza alcanzara el radio de defensa fallida (iteraciones 190 y 196) y en las otras dos no hubo contacto, aunque la distancia mínima fue de 0,052 y 0,056 m, apenas por encima del umbral.

La iteración del primer contacto se distribuyó en dos grupos, 13 simulaciones entre las iteraciones 39 y 40, todas con contacto del defensor 6 y 23 simulaciones entre las iteraciones 59 y 162, con mediana 116 (Figura 14).

En sentido contrario, con un criterio menos exigente, r_c = 0,30 m, todavía inferior a la distancia de contacto físico (≈ 0,42 m; sección 4.1), la proporción de defensas exitosas se mantiene en el 100 % (40 de 40; IC 95 %: 91,2 % – 100 %), con el primer contacto en la iteración 37,7 en promedio (DE 0,7), siempre a cargo del defensor 6. Si el criterio se iguala a la distancia de contacto físico, 0,42 m, el resultado también es del 100 %, con el primer contacto en la iteración 37,2 en promedio. Así, con todos los criterios iguales o inferiores a la condición física de contacto, salvo el de 0,05 m, la proporción de defensas exitosas es del 100 %; solo con este último, mucho más exigente, desciende al 90 % (Tabla 11).

Con el criterio de 0,05 m, el primer contacto se retrasa en la mayoría de las simulaciones y el enjambre se concentra alrededor de la amenaza antes de alcanzarlo. En ese intervalo, en 23 de las 40 simulaciones dos defensores se acercaron a menos de la distancia de contacto físico (≈ 0,42 m) y en 2 de ellas a menos de 0,30 m. Un criterio de contacto más exigente que la condición física no solo reduce la proporción de defensas exitosas, sino que prolonga la fase de convergencia y, con ella, el riesgo de colisiones entre agentes propios.

Indicador	r_c = 0,30 m	r_c = 0,10 m	r_c = 0,05 m
Defensas exitosas	40/40 (100 %)	40/40 (100 %)	36/40 (90,0 %)
IC 95 % (Wilson)	91,2 % – 100 %	91,2 % – 100 %	76,9 % – 96,0 %
Iteración del primer contacto (exitosas)	Media 37,7 (DE 0,7)	Media 38,8 (DE 0,7)	Media 89,0 (DE 43,7); mediana 102
Margen de defensa (exitosas)	Media 131,3 (DE 0,7)	Media 130,2 (DE 0,7)	Media 80,0 (DE 43,7)

Tabla 11. Comparación de resultados según el radio de contacto. Elaboración propia.

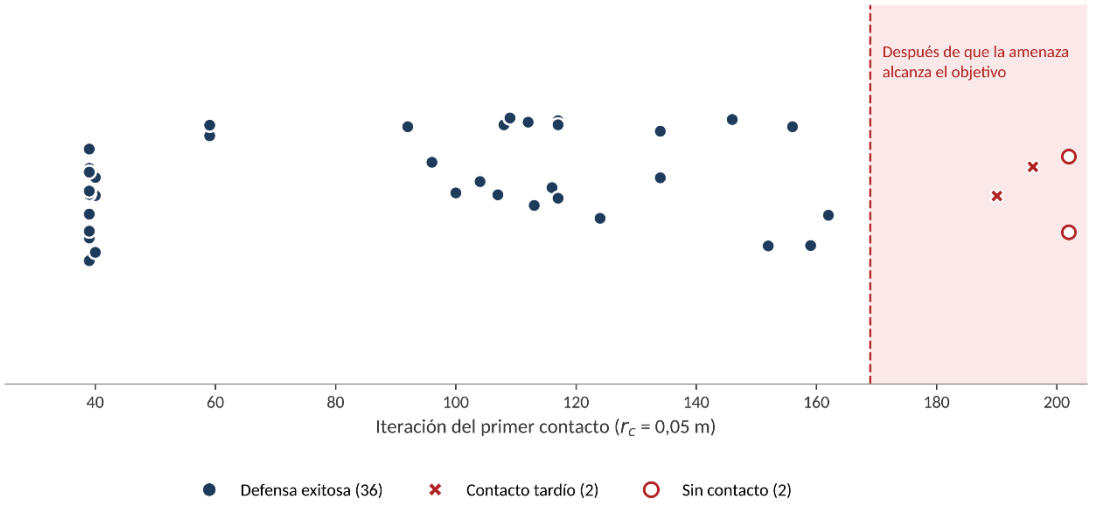


Figura 14. Iteración del primer contacto en las 40 simulaciones con r_c = 0,05 m. Elaboración propia.

4.4. Dimensionamiento del enjambre y análisis de costos

Los resultados anteriores se obtuvieron con un enjambre de ocho defensores. Esta sección justifica ese número a partir de la geometría del despliegue y estima el costo por neutralización del sistema, con el fin de contrastar el argumento de sostenibilidad económica planteado en el marco teórico (sección 1.6).

4.4.1. Número de defensores

En el escenario nominal, la amenaza se detecta desde el inicio de la simulación, a 14 m del activo protegido y avanza 0,08 m por paso, mientras que los defensores pueden desplazarse hasta 0,10 m por paso. Con esa ventaja de tiempo, incluso el defensor más alejado de la trayectoria podría alcanzar la amenaza antes de que llegara al radio de defensa fallida, de modo que el número de defensores no puede justificarse a partir de este escenario, su resultado depende, sobre todo, de que la trayectoria pase por la posición del defensor 6 (sección 4.3.1).

El número de defensores se justifica, en cambio, por el caso más desfavorable, una amenaza que se detecta tarde, cuando cruza el perímetro del anillo defensivo y que puede proceder de cualquier dirección. Para ese caso se calculó, con N defensores distribuidos de forma uniforme sobre el perímetro cuadrado de 10 m de lado y para 3.600 direcciones de entrada, el instante más temprano en que cada defensor podría alcanzar a la amenaza si se desplazara en línea recta, a su velocidad máxima, hacia el punto de encuentro. Se trata de una cota geométrica de alcanzabilidad y no de una simulación del algoritmo, supone que el defensor conoce la trayectoria de la amenaza y alcanza su velocidad máxima de forma instantánea. A partir de ella se obtuvieron dos indicadores para la dirección de entrada más desfavorable, el número de defensores que podrían alcanzar a la amenaza antes de que llegara al radio de defensa fallida, que mide la redundancia y el margen de defensa del defensor más rápido (Figura 15).

Con cuatro defensores, situados en las esquinas, existen direcciones de entrada en las que ninguno puede alcanzar a tiempo a la amenaza, cuando esta entra por el punto medio de un lado, los defensores más cercanos están a 5 m y el margen resulta negativo (−12 iteraciones). Con ocho defensores, uno en cada esquina y uno en el punto medio de cada lado, la separación entre defensores vecinos es de 5 m, ningún punto del perímetro queda a más de 2,5 m de un defensor y al menos cuatro defensores pueden alcanzar a la amenaza desde cualquier dirección, con un margen mínimo de 40 iteraciones. Ese margen es el mayor de todas las configuraciones evaluadas entre 4 y 12 defensores. Con números que no son múltiplos de cuatro, la distribución pierde la simetría del cuadrado y aparecen direcciones de entrada más desfavorables; con 10 o 12 defensores, la redundancia aumenta hasta seis, pero el margen mínimo no mejora (30 y 28 iteraciones) y el costo del enjambre crece en la misma proporción.

Por tanto, con esta geometría, N = 8 es el menor número de defensores que garantiza a la vez una cobertura omnidireccional con margen positivo y una redundancia de al menos cuatro defensores, lo que permite asumir la pérdida de alguno de ellos sin dejar ninguna dirección sin cobertura. Este resultado depende de las velocidades relativas del escenario simulado, de la detección en el perímetro y de una amenaza que se dirige en línea recta al activo. Con amenazas más rápidas que los defensores, ataques simultáneos o perímetros mayores, el número necesario sería distinto, por lo que su verificación mediante co-simulaciones con distintos valores de N se plantea como línea de trabajo futuro.

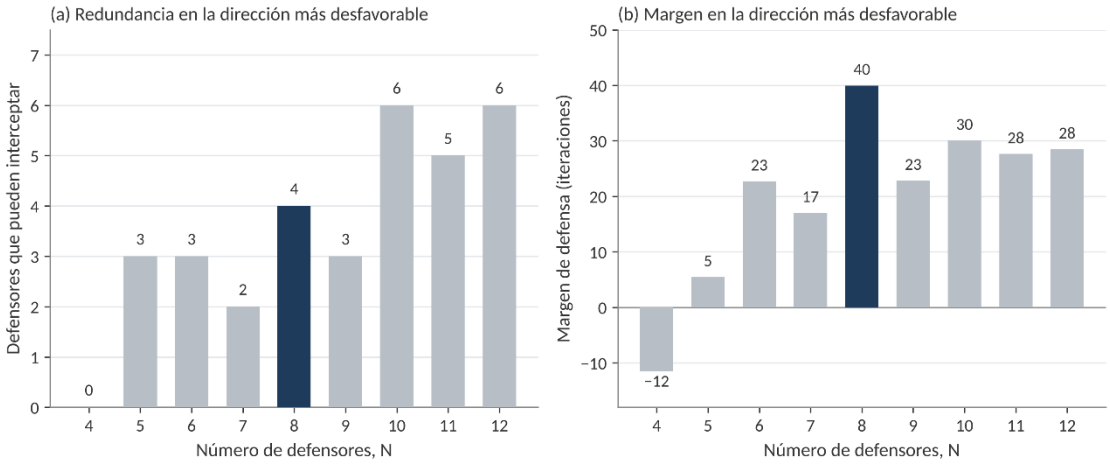


Figura 15. Dimensionamiento del enjambre en la dirección de entrada más desfavorable, con detección en el perímetro, (a) número de defensores que pueden alcanzar a la amenaza antes de que llegue al radio de defensa fallida; (b) margen de defensa del defensor más rápido. Cota geométrica calculada con las velocidades del escenario simulado. Elaboración propia.

4.4.2. Análisis de costos

El argumento económico de la propuesta parte de la asimetría descrita en la sección 1.2.2: en Ucrania se han empleado misiles de entre 1 y más de 3 millones de dólares contra drones de unos 35.000 dólares, es decir, con relaciones de costo de entre 29 y más de 85 a 1 en contra del defensor (Hollenbeck et al., 2025). Para valorar si un enjambre de drones interceptores corrige esa relación, se estimó el costo por neutralización del sistema propuesto mediante un modelo paramétrico sencillo.

En una neutralización cinética, el defensor que establece el contacto se pierde junto con la amenaza, mientras que los demás pueden regresar y reutilizarse. Si cada intento de contacto neutraliza la amenaza con una probabilidad p y los intentos son independientes, el número esperado de defensores consumidos por neutralización es 1/p y el costo por neutralización es $C = c_d / p$, donde c_d es el costo unitario de un defensor. A su vez, la probabilidad de que al menos uno de k defensores logre la neutralización es $1 - (1 - p)^k$. En las simulaciones, la proporción de contactos fue del 100 %, pero la simulación no modela el efecto del impacto, por lo que p no se conoce; por ello se analizan valores de 1, 0,8 y 0,5. Para el costo unitario se adoptan 1.000 y 1.200 dólares, el rango de precios de referencia de un dron de este tipo en el mercado colombiano según la experiencia del autor en el área C-UAS del Ejército Nacional. La Tabla 12 resume los resultados.

p	Defensores consumidos por neutralización (1/p)	Probabilidad de neutralización con 4 defensores	C con c_d = 1.000 USD	C con c_d = 1.200 USD
1,0	1	100 %	1.000 USD	1.200 USD
0,8	1,25	99,8 %	1.250 USD	1.500 USD
0,5	2	93,8 %	2.000 USD	2.400 USD

Tabla 12. Costo estimado por neutralización según la probabilidad de neutralización por intento (p) y el costo unitario del defensor (c_d). Elaboración propia a partir de los valores de referencia indicados en el texto.

Incluso en el supuesto más desfavorable, con un defensor de 1.200 dólares y una probabilidad de neutralización por intento del 50 %, el costo por neutralización, unos 2.400 dólares, es más de dos órdenes de magnitud inferior al de los misiles mencionados. La adquisición del enjambre completo de ocho defensores supondría entre 8.000 y 9.600 dólares, sin contar los repuestos. Además, la redundancia que aporta el dimensionamiento de la sección 4.4.1 se traduce directamente en probabilidad de neutralización, con cuatro defensores capaces de alcanzar la amenaza, la probabilidad de que al menos uno la neutralice supera el 93 % aun con p = 0,5.

La comparación con los sistemas antidrón que se adquieren en Colombia apunta en la misma dirección (Tabla 13). Con una tasa de cambio de referencia de 3.500 pesos por dólar, valor redondeado de la tasa representativa del mercado en 2026, el enjambre de ocho defensores equivale a unos 28 a 34 millones de pesos. Según los valores de referencia aportados por el autor, un sistema antidrón táctico cuesta entre 200 y 800 millones de pesos, es decir, entre 6 y 29 veces más; un sistema semifijo, entre 1.600 y 2.800 millones, entre 50 y 100 veces más y un sistema fijo de altas prestaciones, más de 14.000 millones, más de 400 veces el costo del enjambre. La comparación no es directa, esos sistemas suelen integrar la detección y la neutralización, que habitualmente se basa en la interferencia de radiofrecuencia y no consume el equipo en cada empleo, mientras que el enjambre requiere sensores de detección externos y pierde al menos un defensor por neutralización. Aun así, el orden de magnitud muestra que el enjambre podría añadirse como capa cinética a un sistema existente con un costo marginal reducido.

Sistema	Costo de referencia	Relación con el enjambre
Enjambre de ocho defensores (este trabajo)	8.000–9.600 USD (≈ 28–34 millones COP)	1
Sistema antidrón táctico	200–800 millones COP	6–29 veces
Sistema antidrón semifijo	1.600–2.800 millones COP	50–100 veces
Sistema antidrón fijo de altas prestaciones	Más de 14.000 millones COP	Más de 400 veces

Tabla 13. Costo de adquisición de referencia del enjambre propuesto y de sistemas antidrón en Colombia. Elaboración propia a partir de los valores de referencia aportados por el autor; conversión a 3.500 COP/USD.

Este análisis tiene límites claros. No incluye los costos de los sensores de detección, del sistema de mando y control, de la operación ni del mantenimiento, que en una arquitectura por capas se comparten con el resto de los medios C-UAS, ni los posibles daños por la caída de restos. Tampoco recoge que, como señala Shiffer (2024), el valor de una defensa debe medirse por el activo que protege y no solo por la relación de costos con la amenaza, frente a un ataque contra una unidad militar o una infraestructura crítica, incluso una relación de costos desfavorable puede estar justificada. Con estas salvedades, el resultado respalda que una capa cinética basada en un enjambre de interceptores de bajo costo es compatible con el argumento de sostenibilidad económica, siempre que el costo unitario de los defensores se mantenga en el orden de los miles de dólares.

4.5. Discusión

Los resultados muestran que, en el escenario evaluado, el enjambre estableció contacto con la amenaza en todas las simulaciones y con un amplio margen frente al instante en que esta habría alcanzado su objetivo. El análisis de sensibilidad indica que el resultado se mantiene en un nivel alto aun con un criterio de contacto el doble de estricto, lo que sugiere que no depende exclusivamente del umbral elegido. Además, como el criterio

principal de 0,10 m es más exigente que la condición física de colisión (≈ 0,42 m entre centros), con la que el resultado también es del 100 %, las tasas de éxito obtenidas constituyen una cota conservadora.

Estas cifras deben situarse frente a la literatura. Los trabajos de persecución cooperativa reportan tasas de éxito iguales o superiores al 90 % en escenarios simulados con tres o cinco perseguidores (Ren et al., 2025) y los marcos de interceptación frente a enjambres se evalúan, como este trabajo, con atacantes no evasivos y comunicaciones ideales (Zuo et al., 2026). En ese contexto, el 100 % obtenido no indica que el esquema propuesto supere a esos métodos, sino que el escenario evaluado, con una sola amenaza lenta, no evasiva y detectada con mucha antelación, es menos exigente. La distancia respecto de la experiencia operacional es aún mayor, la defensa ucraniana intercepta entre el 90 % y el 95 % de los drones Shahed de pistón y en torno al 55 % de las variantes a reacción (Anokhin, 2026), cifras que incorporan fallos de detección, de seguimiento y del propio impacto que la simulación no representa. Tampoco es inmediata la comparación de tasas entre estudios, porque, como advierten De Cubber et al. (2025), no existe todavía un conjunto de métricas de neutralización comúnmente aceptado.

Sin embargo, estos resultados deben interpretarse a la luz de la geometría del escenario. La trayectoria rectilínea del dron hostil, desde (−9,9; 9,9) m hacia el activo protegido, pasa por la posición inicial del defensor 6, (−5; 5) m. Por ello, con $r_{\text{?}}$ = 0,10 m, el primer contacto lo estableció siempre ese defensor, entre las iteraciones 38 y 40, en una fase en la que la amenaza atraviesa prácticamente su posición. Este efecto explica también el grupo de contactos tempranos observado con $r_{\text{?}}$ = 0,05 m. En cambio, el grupo de contactos posteriores, entre las iteraciones 59 y 162, refleja el comportamiento cooperativo del enjambre, la convergencia progresiva de varios defensores hacia la amenaza a medida que esta avanza hacia el activo protegido, visible en la Figura 11. La proporción de defensas exitosas debe entenderse, por tanto, como el resultado conjunto de la disposición inicial del enjambre y de su comportamiento colectivo y no como una medida aislada del algoritmo.

Lo mismo ocurre con el tiempo de respuesta. El margen de 130 iteraciones refleja la relación entre las velocidades simuladas y la detección temprana; en el análisis de la sección 4.4.1, con detección en el perímetro, el margen en la dirección más desfavorable se reduce a 40 iteraciones. En los trabajos de guiado terminal (Tao et al., 2022; Pliska et al., 2024), el problema central es precisamente el contrario, alcanzar en pocos segundos a un blanco tan ágil como el interceptor, lo que exige leyes de guiado específicas que el esquema propuesto, basado en la regla de cohesión, no incorpora.

En cuanto a la seguridad, no se registraron colisiones entre defensores antes del primer contacto, en ese intervalo la menor distancia entre ellos fue de 3,77 m. Los acercamientos por debajo de la distancia de contacto físico (≈ 0,42 m) aparecen solo después del choque, en una fase que la simulación no representa de forma realista, o antes del contacto cuando se exige un criterio más estricto (0,05 m), que prolonga la convergencia del enjambre. Este resultado matiza uno de los riesgos identificados en el marco teórico para los sistemas de enjambre cinéticos, las colisiones entre agentes propios, en el escenario evaluado no se materializa antes de la neutralización, pero el radio de separación debería dimensionarse a partir del tamaño real de las plataformas para cubrir también las fases en que el enjambre se concentra sobre la amenaza.

Finalmente, los contactos se determinaron a partir de las posiciones comandadas por MATLAB a los objetos de referencia y no de las posiciones reales de los cuadricópteros en CoppeliaSim. Dado que el controlador de vuelo introduce un retardo en el seguimiento de la referencia, los resultados deben leerse como contactos probables entre trayectorias comandadas. La simulación de validación de la sección 3.5 muestra, sin embargo, que el choque físico con la amenaza se produce en el mismo instante que el primer contacto comandado, por lo que este retardo no altera el instante de neutralización.

Por último, el esquema propuesto difiere de los interceptores empleados en Ucrania en un aspecto relevante. Estos automatizan la mayor parte del ciclo de interceptación, pero mantienen al operador en la selección del objetivo y en la orden de ataque (Ministerio de Defensa de Ucrania, 2026), mientras que el algoritmo evaluado actúa de forma autónoma desde la detección. Esa diferencia no afecta a los resultados de la simulación, pero sí a un eventual empleo operacional, en el que la decisión de neutralizar debería quedar sujeta a control humano. En conjunto, la contribución del trabajo no está en superar el desempeño de los métodos recientes de guiado u optimización, sino en mostrar que un mecanismo de coordinación sencillo, derivado de las reglas de Reynolds, puede organizar un enjambre de interceptores de bajo costo en un entorno con dinámica de vuelo y en ofrecer una plataforma reproducible sobre la cual compararlo con esos métodos.

4.6. Limitaciones

Los resultados deben interpretarse dentro del alcance del trabajo:

- La evaluación se realizó íntegramente en simulación, sobre un área de 20 m × 20 m elegida por su costo computacional.
- Se consideró un único dron hostil, con trayectoria rectilínea no evasiva y un punto de entrada fijo que coincide con la ruta de uno de los defensores.
- Los contactos se calcularon con las posiciones comandadas, no con las posiciones reales de los vehículos y se determinan por proximidad geométrica, sin modelar los efectos físicos de una colisión ni la dispersión de restos.
- La simulación no modela el choque: después del primer contacto, la amenaza y los defensores continúan su movimiento, por lo que los datos posteriores a ese paso no son representativos y no se usan en las métricas.
- La distancia de contacto físico (≈ 0,42 m) se estimó a partir de dos simulaciones de validación y el error de seguimiento, a partir de una de ellas, con posiciones reales registradas solo en el plano horizontal (x, y).
- El radio de la regla de separación se definió sin referencia explícita al tamaño de las plataformas.
- Las velocidades simuladas (≈ 0,08 m/s para la amenaza y ≈ 0,1 m/s como máximo para los defensores) son muy inferiores a las de un UAS real, por lo que los tiempos obtenidos no son extrapolables a tiempos de respuesta reales.
- La integración depende de la Remote API heredada de CoppeliaSim, reemplazada en las versiones recientes del simulador.
- El algoritmo opera de forma autónoma; el control humano sobre la decisión de neutralizar, discutido en el marco teórico, no se modeló.
- El número de defensores se justificó mediante una cota geométrica de alcanzabilidad y el análisis de costos, mediante valores de referencia de costo aportados por el autor y supuestos de probabilidad de neutralización por intento, no mediante simulaciones del algoritmo con distintos valores de N ni datos documentados de adquisición.

CONCLUSIONES

Este trabajo propuso un sistema autónomo de defensa antidrón en el que un enjambre de ocho UAS, coordinado con las reglas de Reynolds, toma como líder al propio dron hostil y actúa como capa cinética final de una arquitectura C-UAS. Los resultados permiten responder a las preguntas de investigación planteadas.

PI1. Sí, en el escenario evaluado. La regla de cohesión dirigida al intruso bastó para interceptar

PI2. Sí, hasta la neutralización. La menor distancia entre defensores antes del primer contacto fue de 3,77 m, lejos de los 0,42 m de contacto físico. Los acercamientos peligrosos solo aparecieron después del choque o con criterios de contacto más estrictos, por lo que el radio de separación debe dimensionarse con el tamaño real de las plataformas.

PI3. Sí. Los nueve cuadricópteros siguieron sus referencias con un error medio de unos 0,10 m, debido en gran parte a un paso de retraso, y en las dos simulaciones de validación el choque físico se produjo en el mismo instante que el primer contacto comandado.

PI4. Con detección tardía en el perímetro, ocho defensores son el menor número que cubre cualquier dirección de entrada con margen positivo y al menos cuatro interceptores disponibles. Con los valores de referencia empleados, el costo por neutralización se sitúa entre 1.000 y 2.400 dólares, frente a los millones de los misiles de defensa aérea.

La hipótesis de trabajo se confirma, por tanto, dentro de los límites del escenario simulado. La Tabla 14 resume el grado de cumplimiento de los objetivos específicos.

Objetivo específico	Cumplimiento	Evidencia
1. Modelo bidimensional del enjambre	Cumplido	Modelo de ocho defensores y una amenaza en un área de 20 m × 20 m, basado en las reglas de Reynolds (Capítulo 2)
2. Algoritmo de control líder-seguidor	Cumplido parcialmente	Cohesión dirigida al dron hostil y adaptación al área protegida; la interacción con el terreno no se abordó
3. Arquitectura de comunicación	Cumplido con otro medio	Integración MATLAB–CoppeliaSim mediante la Remote API (sockets) en lugar de un canal UDP propio; funcionó sin interrupciones en las 40 simulaciones
4. Validación en CoppeliaSim	Cumplido	Seguimiento de las referencias con un error medio de unos 0,10 m; choque físico simultáneo al primer contacto comandado (sección 3.5)
5. Evaluación del desempeño	Cumplido con alcance limitado	Rutas de intercepción y probabilidad de contacto (100 % y 90 %); la probabilidad de derribo no puede estimarse sin modelar el impacto

Tabla 14. Grado de cumplimiento de los objetivos específicos. Elaboración propia.

En conjunto, el objetivo general se cumple en el nivel de una propuesta validada en simulación. El trabajo aporta un esquema de coordinación sencillo, una plataforma de co-simulación reproducible y una evaluación cuantitativa con sus límites explícitos, como punto de partida para estudiar una capa cinética de drones interceptores de bajo costo que no depende del enlace de radiofrecuencia de la amenaza. Su paso a un uso operacional exige las líneas de trabajo que se

proponen a continuación y en particular, mantener el control humano sobre la decisión de neutralizar.

Líneas de trabajos futuros

Los resultados y las limitaciones identificadas permiten proponer las siguientes líneas de trabajo:

- **Escenarios más exigentes:** varios puntos de entrada, amenazas evasivas, más rápidas que los defensores o simultáneas, y un terreno con relieve y obstáculos, componente del segundo objetivo que no se abordó.
- **Dimensionamiento del enjambre:** co-simulaciones con distintos números de defensores (de 4 a 12) y detección tardía, para contrastar la cota geométrica de la sección 4.4.1.
- **Percepción y seguridad:** radio de percepción limitado, retardos y pérdidas en las comunicaciones, errores de detección y un radio de separación dimensionado con el tamaño real de las plataformas.
- **Comparación y efecto del impacto:** comparar el esquema con leyes de guiado terminal y de persecución cooperativa (Tao et al., 2022; Ren et al., 2025; Xu et al., 2026) con las métricas de De Cubber et al. (2025), migrar a la ZeroMQ Remote API y modelar el impacto para estimar una probabilidad de derribo.
- **Validación experimental y empleo:** ensayos con vehículos reales, costos documentados de adquisición en Colombia y análisis del marco legal y ético, en particular del control humano sobre la decisión de neutralizar.

Bibliografía

Abdelkader, M., Güler, S., Jaleel, H. y Shamma, J. S. (2021). Aerial swarms: Recent applications and challenges. *Current Robotics Reports, 2*, 309–320. https://doi.org/10.1007/s43154-021-00063-4

Anokhin, I. (2026, 13 de octubre). *Monthly analysis of Russian Shahed 136 deployment against Ukraine*. Institute for Science and International Security. https://isis-online.org/isis-reports/monthly-analysis-of-russian-shahed-136-deployment-against-ukraine

Brambilla, M., Ferrante, E., Birattari, M. y Dorigo, M. (2013). Swarm robotics: A review from the swarm engineering perspective. *Swarm Intelligence, 7*(1), 1–41. https://doi.org/10.1007/s11721-012-0075-2

Brust, M. R., Danoy, G., Stolfi, D. H. y Bouvry, P. (2021). Swarm-based counter UAV defense system. *Discover Internet of Things, 1*, Artículo 2. https://doi.org/10.1007/s43926-021-00002-x

Calcara, A., Gilli, A., Gilli, M., Marchetti, R. y Zaccagnini, I. (2022). Why drones have not revolutionized war: The enduring hider-finder competition in air warfare. *International Security, 46*(4), 130–171. https://doi.org/10.1162/isec_a_00431

Cardona, G. A. y Calderón, J. M. (2019). Robot swarm navigation and victim detection using rendezvous consensus in search and rescue operations. *Applied Sciences, 9*(8), 1702. https://doi.org/10.3390/app9081702

Castrillo, V. U., Manco, A., Pascarella, D. y Gigante, G. (2022). A review of counter-UAS technologies for cooperative defensive teams of drones. *Drones, 6*(3), 65. https://doi.org/10.3390/drones6030065

Chamola, V., Kotesh, P., Agarwal, A., Naren, Gupta, N. y Guizani, M. (2021). A comprehensive review of unmanned aerial vehicle attacks and neutralization techniques. *Ad Hoc Networks, 111*, 102324. https://doi.org/10.1016/j.adhoc.2020.102324

De Cubber, G., Doroftei, D., Petsioti, P., Koniaris, A., Brewczyński, K., Życzkowski, M., Roman, R., Sima, S., Mohamoud, A., van de Pol, J., Maza, I., Ollero, A., Church, C. y Popa, C. (2025). Standardized evaluation of counter-drone systems: Methods, technologies, and performance metrics. *Drones, 9*(5), 354. https://doi.org/10.3390/drones9050354

Elamvazhuthi, K., Kakish, Z., Shirsat, A. y Berman, S. (2021). Controllability and stabilization for herding a robotic swarm using a leader: A mean-field approach. *IEEE Transactions on Robotics, 37*(2), 418–432. https://doi.org/10.1109/TRO.2020.3031237

Gao, Y., Bai, C., Zhang, L. y Quan, Q. (2022). Multi-UAV cooperative target encirclement within an annular virtual tube. *Aerospace Science and Technology, 128*, 107800. https://doi.org/10.1016/j.ast.2022.107800

Gettinger, D. M. (2024, 25 de octubre). *Defense primer: Categories of uncrewed aircraft systems* (IF12797). Congressional Research Service. https://www.congress.gov/crs_external_products/IF/PDF/IF12797/IF12797.2.pdf

González-Jorge, H., Aldao, E., Fontenla-Carrera, G., Veiga-López, F., Balvís, E. y Ríos-Otero, E. (2024). Counter drone technology. *DYNA – Ingeniería e Industria, 99*(6). https://doi.org/10.52152/D11227

Hollenbeck, N., Altaf, M. H., Avila, F., Ramirez, J., Sharma, A. y Jensen, B. (2025, 19 de febrero). *Calculating the cost-effectiveness of Russia's drone strikes*. Center for Strategic and International Studies. https://www.csis.org/analysis/calculating-cost-effectiveness-russias-drone-strikes

Joint Air Power Competence Centre. (2010). *Strategic concept of employment for unmanned aircraft systems in NATO*. JAPCC. https://www.japcc.org/wp-content/uploads/UAS_CONEMP.pdf

Kang, H., Joung, J., Kim, J., Kang, J. y Cho, Y. S. (2020). Protect your sky: A survey of counter unmanned aerial vehicle systems. *IEEE Access, 8*, 168671–168710. https://doi.org/10.1109/ACCESS.2020.3023473

Kunertova, D. (2023). The war in Ukraine shows the game-changing effect of drones depends on the game. *Bulletin of the Atomic Scientists, 79*(2), 95–102. https://doi.org/10.1080/00963402.2023.2178180

León, J., Cardona, G. A., Jaimes, L. G., Calderón, J. M. y Ospina Rodríguez, P. (2018). Rendezvous consensus algorithm applied to the location of possible victims in disaster zones. En L. Rutkowski, R. Scherer, M. Korytkowski, W. Pedrycz, R. Tadeusiewicz y J. M. Zurada (Eds.), *Artificial intelligence and soft computing* (Lecture Notes in Computer Science, Vol. 10842, pp. 700–710). Springer. https://doi.org/10.1007/978-3-319-91262-2_61

Lewis, F. L., Zhang, H., Hengster-Movric, K. y Das, A. (2014). *Cooperative control of multi-agent systems: Optimal and adaptive design approaches*. Springer. https://doi.org/10.1007/978-1-4471-5574-4

Ministerio de Defensa de Ucrania. (2026, 8 de abril). *A record 33,000 enemy UAVs were destroyed by interceptor drones in March, double the number recorded in the previous month*. https://mod.gov.ua/en/news/a-record-33-000-enemy-ua-vs-were-destroyed-by-interceptor-drones-in-march-double-the-number-recorded-in-the-previous-month

Ministerio de Defensa de Ucrania. (2026, 8 de junio). *Next-generation interceptors: Ukrainian drones already autonomously take down Shahed-type UAVs*.

https://mod.gov.ua/en/news/next-generation-interceptors-ukrainian-drones-already-autonomously-take-down-shahed-type-ua-vs

Newaz, A. A. R., Lee, G., Pratama, F. A. y Chong, N. Y. (2013). 3D exploration priority based flocking of UAVs. En *2013 IEEE International Conference on Mechatronics and Automation*. IEEE. https://doi.org/10.1109/ICMA.2013.6618142

Pliska, M., Vrba, M., Báča, T. y Saska, M. (2024). Towards safe mid-air drone interception: Strategies for tracking & capture. *IEEE Robotics and Automation Letters, 9*(10), 8810–8817. https://doi.org/10.1109/LRA.2024.3451768

Prieto-Méndez, D. A. y Pirateque-Perdomo, P. (2025). Desafíos de seguridad y defensa de Colombia ante UAV en conflictos modernos. *Inteligencia Estratégica, 2*(2), 137–155. https://revista.esici.edu.co/index.php/inest/article/view/26

Ren, H., Han, C., Pan, H., Sun, J., Li, S., An, D. y Hu, K. (2025). Multi-UAV cooperative pursuit planning via communication-aware multi-agent reinforcement learning. *Aerospace, 12*(11), 993. https://doi.org/10.3390/aerospace12110993

Ren, W. y Beard, R. W. (2008). *Distributed consensus in multi-vehicle cooperative control: Theory and applications*. Springer. https://doi.org/10.1007/978-1-84800-015-5

Reynolds, C. W. (1987). Flocks, herds, and schools: A distributed behavioral model. *Computer Graphics, 21*(4), 25–34. https://doi.org/10.1145/37401.37406

Rohmer, E., Singh, S. P. N. y Freese, M. (2013). V-REP: A versatile and scalable robot simulation framework. En *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems* (pp. 1321–1326). IEEE. https://doi.org/10.1109/IROS.2013.6696520

Servicio de las Naciones Unidas de Actividades Relativas a las Minas [UNMAS]. (2026). *Sistemas de aeronaves no tripuladas (UAS) con cargas explosivas: amenazas y desafíos emergentes para Colombia*. Naciones Unidas. https://unsco.unmissions.org/sites/default/files/2026-06/UNMAS%20Colombia%20UAS_UAV%20ESP%202026.pdf

Shiffer, A. (2024, diciembre). *Calculating the true value of air defence*. Joint Air Power Competence Centre. https://www.japcc.org/?p=15974

Souli, N., Kolios, P. y Ellinas, G. (2023). Multi-agent system for rogue drone interception. *IEEE Robotics and Automation Letters, 8*(4), 2221–2228. https://doi.org/10.1109/LRA.2023.3245412

Tao, H., Lin, D., He, S., Song, T. y Jin, R. (2022). Optimal terminal-velocity-control guidance for intercepting non-cooperative maneuvering quadcopter. *Journal of Field Robotics, 39*(4), 457–472. https://doi.org/10.1002/rob.22059

Wang, C., Shi, Z., Gu, M., Luo, W., Zhu, X. y Fan, Z. (2024). Revolutionary entrapment model of uniformly distributed swarm robots in morphogenetic formation. *Defence Technology, 31*, 496–509. https://doi.org/10.1016/j.dt.2022.08.015

Wang, J., Liu, Y. y Song, H. (2021). Counter-unmanned aircraft system(s) (C-UAS): State of the art, challenges, and future trends. *IEEE Aerospace and Electronic Systems Magazine, 36*(3), 4–29. https://doi.org/10.1109/MAES.2020.3015537

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Xie, Y., Han, L., Zhang, L., Dong, X., Li, Q. y Ren, Z. (2020). Adaptive formation tracking control for UAV swarm systems with multiple leaders and switching topologies. En *2020 Chinese Automation Congress (CAC)* (pp. 5532–5537). IEEE. https://doi.org/10.1109/CAC51589.2020.9327412

Xu, J., Tao, H. y Lin, D. (2026). Distributed optimal spatial–temporal cooperative pursuit guidance for multi-UAV interception. *Aerospace, 13*(6), 542. https://doi.org/10.3390/aerospace13060542

Zuo, L., Wang, Y., Liu, J., Lu, Y. y Gu, R. (2026). A hierarchical cooperative interception framework for multi-UAV defense against large-scale swarm intrusions. *Drones, 10*(6), 418. https://doi.org/10.3390/drones10060418

Anexo A. Código QuadcopterMove.m

Script principal de la co-simulación. Inicializa el escenario, se conecta con CoppeliaSim mediante la Remote API heredada, calcula en cada iteración la posición del dron hostil y las reglas de cohesión, separación, alineación y evitación de los defensores, envía las posiciones comandadas a los objetos target de la escena y registra las posiciones reales de los cuadricópteros. Al finalizar, llama al módulo de registro de datos (Anexo B).

```
%%
clc
num_defeders = 8;

range_x                     = [-10, 10];
range_y                     = [-10, 10];
boidAlt                     = 4;
num_iterations              = 200;
cohesion_factor             = 0.02;
separation_factor           = 0.05;
alignment_factor            = 0.01;
Enemy_avoidance_factor      = 0.01;
Enemy_separation_distance   = 0.01;

pos_ini_enemy               = [-9.9, 9.9];
enemy_obj                   = [0,  0];
enemy_Vel                   = 0.08;
Enemy                       = pos_ini_enemy;

defenders = [+5  -0;   %Quadcopter[0]
             +5  -5;   %Quadcopter[1]
             -0  -5;   %Quadcopter[2]
             -5  -5;   %Quadcopter[3]
             -5  +0;   %Quadcopter[4]
             -5  +5;   %Quadcopter[5]
             +0  +5;   %Quadcopter[6]
             +5  +5];  %Quadcopter[7]

trajectory01 = zeros(num_iterations,3);
trajectory02 = zeros(num_iterations,3);
trajectory03 = zeros(num_iterations,3);
trajectory04 = zeros(num_iterations,3);
trajectory05 = zeros(num_iterations,3);
trajectory06 = zeros(num_iterations,3);
trajectory07 = zeros(num_iterations,3);
trajectory08 = zeros(num_iterations,3);
trajectory09 = zeros(num_iterations,3);

realpos01    = zeros(num_iterations,3);
realpos02    = zeros(num_iterations,3);
realpos03    = zeros(num_iterations,3);
realpos04    = zeros(num_iterations,3);
realpos05    = zeros(num_iterations,3);
realpos06    = zeros(num_iterations,3);
realpos07    = zeros(num_iterations,3);
realpos08    = zeros(num_iterations,3);
realpos09    = zeros(num_iterations,3);

velocity_D = [0, 0];
velocities = rand(num_defeders, 2) - 0.5;

% Conectar a CoppeliaSim
sim=remApi('remoteApi'); % using the prototype file (remoteApiProto.m)
sim.simxFinish(-1); % just in case, close all opened connections
clientID=sim.simxStart('127.0.0.1',19999,true,true,5000,5);

% Verificar la conexión
if (clientID > -1)
    disp('Conectado a CoppeliaSim');

    % Obtener el handle del target del cuadricóptero
    [res1, target_handle1] = sim.simxGetObjectHandle (clientID,
    ':/target[0]',sim.simx_opmode_blocking);
    [res2, target_handle2] = sim.simxGetObjectHandle (clientID,
    ':/target[1]',sim.simx_opmode_blocking);
    [res3, target_handle3] = sim.simxGetObjectHandle (clientID,
    ':/target[2]',sim.simx_opmode_blocking);
```

```
[res4, target_handle4] = sim.simxGetObjectHandle (clientID,
':/target[3]',sim.simx_opmode_blocking);
[res5, target_handle5] = sim.simxGetObjectHandle (clientID,
':/target[4]',sim.simx_opmode_blocking);
[res6, target_handle6] = sim.simxGetObjectHandle (clientID,
':/target[5]',sim.simx_opmode_blocking);
[res7, target_handle7] = sim.simxGetObjectHandle (clientID,
':/target[6]',sim.simx_opmode_blocking);
[res8, target_handle8] = sim.simxGetObjectHandle (clientID,
':/target[7]',sim.simx_opmode_blocking);
[res9, target_handle9] = sim.simxGetObjectHandle (clientID,
':/target[8]',sim.simx_opmode_blocking);


[resB1, base_handle1] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[0]',sim.simx_opmode_blocking);
[resB2, base_handle2] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[1]',sim.simx_opmode_blocking);
[resB3, base_handle3] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[2]',sim.simx_opmode_blocking);
[resB4, base_handle4] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[3]',sim.simx_opmode_blocking);
[resB5, base_handle5] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[4]',sim.simx_opmode_blocking);
[resB6, base_handle6] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[5]',sim.simx_opmode_blocking);
[resB7, base_handle7] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[6]',sim.simx_opmode_blocking);
[resB8, base_handle8] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[7]',sim.simx_opmode_blocking);
[resB9, base_handle9] = sim.simxGetObjectHandle (clientID,
':/Quadcopter[8]',sim.simx_opmode_blocking);

target_pos1 = [Enemy(1,1), Enemy(1,2), boidAlt];
target_pos2 = [defenders(1,1), defenders(1,2), boidAlt];
target_pos3 = [defenders(2,1), defenders(2,2), boidAlt];
target_pos4 = [defenders(3,1), defenders(3,2), boidAlt];
target_pos5 = [defenders(4,1), defenders(4,2), boidAlt];
target_pos6 = [defenders(5,1), defenders(5,2), boidAlt];
target_pos7 = [defenders(6,1), defenders(6,2), boidAlt];
target_pos8 = [defenders(7,1), defenders(7,2), boidAlt];
target_pos9 = [defenders(8,1), defenders(8,2), boidAlt];


if (resB1 == sim.simx_return_ok && ...
    resB2 == sim.simx_return_ok && ...
    resB3 == sim.simx_return_ok && ...
    resB4 == sim.simx_return_ok && ...
    resB5 == sim.simx_return_ok && ...
    resB6 == sim.simx_return_ok && ...
    resB7 == sim.simx_return_ok && ...
    resB8 == sim.simx_return_ok && ...
    resB9 == sim.simx_return_ok)
    for i=1:20


            [res1]  = sim.simxSetObjectPosition(clientID, target_handle1,  -1, target_pos1,
sim.simx_opmode_continuous);
            [res2]  = sim.simxSetObjectPosition(clientID, target_handle2,  -1, target_pos2,
sim.simx_opmode_continuous);
            [res3]  = sim.simxSetObjectPosition(clientID, target_handle3,  -1, target_pos3,
sim.simx_opmode_continuous);
            [res4]  = sim.simxSetObjectPosition(clientID, target_handle4,  -1, target_pos4,
sim.simx_opmode_continuous);
            [res5]  = sim.simxSetObjectPosition(clientID, target_handle5,  -1, target_pos5,
sim.simx_opmode_continuous);
            [res6]  = sim.simxSetObjectPosition(clientID, target_handle6,  -1, target_pos6,
sim.simx_opmode_continuous);
            [res7]  = sim.simxSetObjectPosition(clientID, target_handle7,  -1, target_pos7,
sim.simx_opmode_continuous);
            [res8]  = sim.simxSetObjectPosition(clientID, target_handle8,  -1, target_pos8,
sim.simx_opmode_continuous);
            [res9]  = sim.simxSetObjectPosition(clientID, target_handle9,  -1, target_pos9,
sim.simx_opmode_continuous);

            [resB1, RPQuadcopter0] = sim.simxGetObjectPosition(clientID, base_handle1, -1,
sim.simx_opmode_continuous);
            [resB2, RPQuadcopter1] = sim.simxGetObjectPosition(clientID, base_handle2, -1,
sim.simx_opmode_continuous);
```

```
            [resB3, RPQuadcopter2] = sim.simxGetObjectPosition(clientID, base_handle3, -1,
sim.simx_opmode_continuous);
            [resB4, RPQuadcopter3] = sim.simxGetObjectPosition(clientID, base_handle4, -1,
sim.simx_opmode_continuous);
            [resB5, RPQuadcopter4] = sim.simxGetObjectPosition(clientID, base_handle5, -1,
sim.simx_opmode_continuous);
            [resB6, RPQuadcopter5] = sim.simxGetObjectPosition(clientID, base_handle6, -1,
sim.simx_opmode_continuous);
            [resB7, RPQuadcopter6] = sim.simxGetObjectPosition(clientID, base_handle7, -1,
sim.simx_opmode_continuous);
            [resB8, RPQuadcopter7] = sim.simxGetObjectPosition(clientID, base_handle8, -1,
sim.simx_opmode_continuous);
            [resB9, RPQuadcopter8] = sim.simxGetObjectPosition(clientID, base_handle9, -1,
sim.simx_opmode_continuous);
    end
end


if (res1 == sim.simx_return_ok && ...
    res2 == sim.simx_return_ok && ...
    res3 == sim.simx_return_ok && ...
    res4 == sim.simx_return_ok && ...
    res5 == sim.simx_return_ok && ...
    res6 == sim.simx_return_ok && ...
    res6 == sim.simx_return_ok && ...
    res7 == sim.simx_return_ok && ...
    res8 == sim.simx_return_ok && ...
    res9 == sim.simx_return_ok)

    disp('Handle del target obtenido correctamente');

   for t = 1:num_iterations
       tic;
       d    = enemy_obj - Enemy;
       dist = norm(d);

       if dist > 1e-9
           Enemy = Enemy + d / dist * min(enemy_Vel, dist);
       end

        for i = 1:num_defeders
            % Regla de cohesion enjambre defensor
            cohesion = (Enemy  - defenders(i,:))*cohesion_factor;

            %Regla Separacion Enjambre Defensor
            separation = zeros(1,2);
            for j = 1:num_defeders
                if i ~= j
                    dist = norm(defenders(i,:) - defenders(j,:));
                    if dist < 0.1                                       % Evitar división por cero
                        separation = separation + rand(1,2);            % Aplicar un pequeño
alejamiento aleatorio
                    elseif dist < 0.5
                        separation = separation - (defenders(j,:) - defenders(i,:)) / dist^2;
                    end
                end
            end
            separation = separation * separation_factor;


            % Regla de separación de lider
            enemy_avoidance = zeros(1,2);
            for k = 1:size(Enemy, 1)
                dist_to_Enemy = norm(defenders(i,:) - Enemy);
                if dist_to_Enemy < Enemy_separation_distance
                    enemy_avoidance = enemy_avoidance + (defenders(i,:) - Enemy) / dist_to_Enemy^2;
                end
            end
            enemy_avoidance = enemy_avoidance * Enemy_avoidance_factor;

            % REGLA DE ALINEACIÓN
            alignment = mean(velocities) * alignment_factor;


             % Actualizar velocidad
            velocities(i,:) = velocities(i,:) + cohesion + separation + alignment +
enemy_avoidance;
            %
            max_speed = 0.1;
```

```
            speed = norm(velocities(i,:));
            if speed > max_speed
                velocities(i,:) = velocities(i,:) * max_speed / speed;
            end



             % Actualizar posición defensores
            defenders(i,:) = defenders(i,:) + velocities(i,:);


            target_pos1 = [Enemy(1,1), Enemy(1,2), boidAlt];
            target_pos2 = [defenders(1,1), defenders(1,2), boidAlt];
            target_pos3 = [defenders(2,1), defenders(2,2), boidAlt];
            target_pos4 = [defenders(3,1), defenders(3,2), boidAlt];
            target_pos5 = [defenders(4,1), defenders(4,2), boidAlt];
            target_pos6 = [defenders(5,1), defenders(5,2), boidAlt];
            target_pos7 = [defenders(6,1), defenders(6,2), boidAlt];
            target_pos8 = [defenders(7,1), defenders(7,2), boidAlt];
            target_pos9 = [defenders(8,1), defenders(8,2), boidAlt];


            [res1]  = sim.simxSetObjectPosition(clientID, target_handle1,  -1, target_pos1,
sim.simx_opmode_continuous);
            [res2]  = sim.simxSetObjectPosition(clientID, target_handle2,  -1, target_pos2,
sim.simx_opmode_continuous);
            [res3]  = sim.simxSetObjectPosition(clientID, target_handle3,  -1, target_pos3,
sim.simx_opmode_continuous);
            [res4]  = sim.simxSetObjectPosition(clientID, target_handle4,  -1, target_pos4,
sim.simx_opmode_continuous);
            [res5]  = sim.simxSetObjectPosition(clientID, target_handle5,  -1, target_pos5,
sim.simx_opmode_continuous);
            [res6]  = sim.simxSetObjectPosition(clientID, target_handle6,  -1, target_pos6,
sim.simx_opmode_continuous);
            [res7]  = sim.simxSetObjectPosition(clientID, target_handle7,  -1, target_pos7,
sim.simx_opmode_continuous);
            [res8]  = sim.simxSetObjectPosition(clientID, target_handle8,  -1, target_pos8,
sim.simx_opmode_continuous);
            [res9]  = sim.simxSetObjectPosition(clientID, target_handle9,  -1, target_pos9,
sim.simx_opmode_continuous);

            [resB1, RPQuadcopter0] = sim.simxGetObjectPosition(clientID, base_handle1, -1,
sim.simx_opmode_continuous);
            [resB2, RPQuadcopter1] = sim.simxGetObjectPosition(clientID, base_handle2, -1,
sim.simx_opmode_continuous);
            [resB3, RPQuadcopter2] = sim.simxGetObjectPosition(clientID, base_handle3, -1,
sim.simx_opmode_continuous);
            [resB4, RPQuadcopter3] = sim.simxGetObjectPosition(clientID, base_handle4, -1,
sim.simx_opmode_continuous);
            [resB5, RPQuadcopter4] = sim.simxGetObjectPosition(clientID, base_handle5, -1,
sim.simx_opmode_continuous);
            [resB6, RPQuadcopter5] = sim.simxGetObjectPosition(clientID, base_handle6, -1,
sim.simx_opmode_continuous);
            [resB7, RPQuadcopter6] = sim.simxGetObjectPosition(clientID, base_handle7, -1,
sim.simx_opmode_continuous);
            [resB8, RPQuadcopter7] = sim.simxGetObjectPosition(clientID, base_handle8, -1,
sim.simx_opmode_continuous);
            [resB9, RPQuadcopter8] = sim.simxGetObjectPosition(clientID, base_handle9, -1,
sim.simx_opmode_continuous);


            trajectory01(t,:) = [Enemy(1,:),    boidAlt];
            trajectory02(t,:) = [defenders(1,:),boidAlt];
            trajectory03(t,:) = [defenders(2,:),boidAlt];
            trajectory04(t,:) = [defenders(3,:),boidAlt];
            trajectory05(t,:) = [defenders(4,:),boidAlt];
            trajectory06(t,:) = [defenders(5,:),boidAlt];
            trajectory07(t,:) = [defenders(6,:),boidAlt];
            trajectory08(t,:) = [defenders(7,:),boidAlt];
            trajectory09(t,:) = [defenders(8,:),boidAlt];


            realpos01(t,:) = RPQuadcopter0;
            realpos02(t,:) = RPQuadcopter1;
            realpos03(t,:) = RPQuadcopter2;
            realpos04(t,:) = RPQuadcopter3;
            realpos05(t,:) = RPQuadcopter4;
            realpos06(t,:) = RPQuadcopter5;
            realpos07(t,:) = RPQuadcopter6;
```

```
                realpos08(t,:) = RPQuadcopter7;
                realpos09(t,:) = RPQuadcopter8;

            end


            if (res1 == sim.simx_return_ok && ...
                res2 == sim.simx_return_ok && ...
                res3 == sim.simx_return_ok && ...
                res4 == sim.simx_return_ok && ...
                res5 == sim.simx_return_ok && ...
                res6 == sim.simx_return_ok && ...
                res6 == sim.simx_return_ok && ...
                res7 == sim.simx_return_ok && ...
                res8 == sim.simx_return_ok && ...
                res9 == sim.simx_return_ok)

                disp(['Posición del target establecida correctamente -- ', num2str(t)]);
                pause(1)
                tiempoIteracion = toc;
            else
                disp('Error al establecer la posición del target');
            end
       end
    else
        disp('Error al obtener el handle del target');
    end

    % Detener la simulación y cerrar la conexión con coppelia
    sim.simxStopSimulation(clientID, sim.simx_opmode_blocking);
    sim.simxFinish(clientID);
    sim.delete();
else
    disp('Error al conectar con CoppeliaSim');
end
    disp('Simulación detenida y conexión cerrada');
    Datalogger
close all;
```

Anexo B. Código Datalogger.m

Módulo de registro de datos. Genera un archivo CSV con la iteración y las posiciones comandadas y reales, en el plano (x, y), del dron hostil y de los ocho defensores. Se ejecuta al final de QuadcopterMove.m y utiliza las variables que este deja en el espacio de trabajo de MATLAB.

```
% ========================================
% CREAR CSV DE LA SIMULACION
% ========================================

carpeta = 'C:\Users\User\Documents\pythonPrj\resultados_matlab-coppelia';

% Fecha y hora de creacion
fechaHora = datetime('now', ...
                     'Format', ...
                     'yyyyMMdd_HHmmss');

% Nombre del archivo
nombreArchivo = ['Simulacion_Enjambre_' ...
                 char(fechaHora) '.csv'];

rutaArchivo = fullfile(carpeta, nombreArchivo);

% Numero de drones del enjambre
numDrones = 8;

% Crear encabezados
encabezados = {'Iteracion', 'Enemy_target x', 'Enemy_target y','Enemy_real x', 'Enemy Real y'};

for k = 1:numDrones
    encabezados{end+1} = sprintf('Def_target x %d', k);
    encabezados{end+1} = sprintf('Def_target y %d', k);
    encabezados{end+1} = sprintf('Def_real x %d', k);
    encabezados{end+1} = sprintf('Def_real y %d', k);
end

for k = 1:numDrones


end

% Abrir archivo CSV
fid = fopen(rutaArchivo, 'w');

if fid == -1
    error('No se pudo crear el archivo CSV.');
end

% Escribir encabezados
fprintf(fid, '%s', encabezados{1});

for k = 2:length(encabezados)
    fprintf(fid, ',%s', encabezados{k});
end
fprintf(fid, '\n');

% Inicializar coordenadas de los 8 drones
swarmXY= [+5  -0;   %Quadcopter[0]
          +5  -5;   %Quadcopter[1]
          -0  -5;   %Quadcopter[2]
          -5  -5;   %Quadcopter[3]
          -5  +0;   %Quadcopter[4]
          -5  +5;   %Quadcopter[5]
          +0  +5;   %Quadcopter[6]
          +5  +5];  %Quadcopter[7]

swarmXY_R= [+5  -0;   %Quadcopter[0]
            +5  -5;   %Quadcopter[1]
            -0  -5;   %Quadcopter[2]
            -5  -5;   %Quadcopter[3]
            -5  +0;   %Quadcopter[4]
            -5  +5;   %Quadcopter[5]
            +0  +5;   %Quadcopter[6]
            +5  +5];  %Quadcopter[7]
```

```
for i=1:t
    iteracion = i;

     enemyX = trajectory01(i,1);
     enemyY = trajectory01(i,2);

     enemyX_R = realpos01(i,1);
     enemyY_R = realpos01(i,2);

     swarmXY(1,1)= trajectory02(i,1);
     swarmXY(1,2)= trajectory02(i,2);

     swarmXY(2,1)= trajectory03(i,1);
     swarmXY(2,2)= trajectory03(i,2);

     swarmXY(3,1)= trajectory04(i,1);
     swarmXY(3,2)= trajectory04(i,2);

     swarmXY(4,1)= trajectory05(i,1);
     swarmXY(4,2)= trajectory05(i,2);

     swarmXY(5,1)= trajectory06(i,1);
     swarmXY(5,2)= trajectory06(i,2);

     swarmXY(6,1)= trajectory07(i,1);
     swarmXY(6,2)= trajectory07(i,2);

     swarmXY(7,1)= trajectory08(i,1);
     swarmXY(7,2)= trajectory08(i,2);

     swarmXY(8,1)= trajectory09(i,1);
     swarmXY(8,2)= trajectory09(i,2);

     swarmXY_R(1,1)= realpos02(i,1);
     swarmXY_R(1,2)= realpos02(i,2);

     swarmXY_R(2,1)= realpos03(i,1);
     swarmXY_R(2,2)= realpos03(i,2);

     swarmXY_R(3,1)= realpos04(i,1);
     swarmXY_R(3,2)= realpos04(i,2);

     swarmXY_R(4,1)= realpos05(i,1);
     swarmXY_R(4,2)= realpos05(i,2);

     swarmXY_R(5,1)= realpos06(i,1);
     swarmXY_R(5,2)= realpos06(i,2);

     swarmXY_R(6,1)= realpos07(i,1);
     swarmXY_R(6,2)= realpos07(i,2);

     swarmXY_R(7,1)= realpos08(i,1);
     swarmXY_R(7,2)= realpos08(i,2);

     swarmXY_R(8,1)= realpos09(i,1);
     swarmXY_R(8,2)= realpos09(i,2);


    fila = [iteracion, ...
            enemyX, enemyY, enemyX_R, enemyY_R...
            swarmXY(1,1), swarmXY(1,2), swarmXY_R(1,1), swarmXY_R(1,2)...
            swarmXY(2,1), swarmXY(2,2), swarmXY_R(2,1), swarmXY_R(2,2)...
            swarmXY(3,1), swarmXY(3,2), swarmXY_R(3,1), swarmXY_R(3,2)...
            swarmXY(4,1), swarmXY(4,2), swarmXY_R(4,1), swarmXY_R(4,2)...
            swarmXY(5,1), swarmXY(5,2), swarmXY_R(5,1), swarmXY_R(5,2)...
            swarmXY(6,1), swarmXY(6,2), swarmXY_R(6,1), swarmXY_R(6,2)...
            swarmXY(7,1), swarmXY(7,2), swarmXY_R(7,1), swarmXY_R(7,2)...
            swarmXY(8,1), swarmXY(8,2), swarmXY_R(8,1), swarmXY_R(8,2)];

    fprintf(fid, '%d', fila(1));
    fprintf(fid, ',%.4f', fila(2:end));
    fprintf(fid, '\n');


end

% Cerrar archivo
fclose(fid);
```

```
fprintf('Archivo creado: %s\n', rutaArchivo);
```

Anexo C. Script Lua de la escena de CoppeliaSim

Script Lua asociado a un objeto de la escena de CoppeliaSim. En la inicialización arranca el servidor de la Remote API heredada en el puerto 19999, al que se conecta QuadcopterMove.m (Anexo A), y obtiene los identificadores de los nueve objetos target y de los nueve cuadricópteros mediante sim.getObject, la función de acceso a objetos de la versión del simulador empleada (sección 3.2). Las funciones de actuación y de sensado quedan vacías, porque la posición de los objetos target la impone MATLAB en cada paso y los controladores de vuelo de los cuadricópteros se encargan de seguirla.

```
function sysCall_init()
    -- do some initialization here
    -- Codigo de Interaccion con Matlab
       simRemoteApi.start(19999)

       targetHandle0   = sim.getObject("/target[0]") -- dron Eemigo
       targetHandle1   = sim.getObject("/target[1]") -- Aliado1 [ 5  0]
       targetHandle2   = sim.getObject("/target[2]") -- Aliado2 [ 5 -5]
       targetHandle3   = sim.getObject("/target[3]") -- Aliado3 [ 0  5]
       targetHandle4   = sim.getObject("/target[4]") -- Aliado4 [-5 -5]
       targetHandle5   = sim.getObject("/target[5]") -- Aliado5 [-5  0]
       targetHandle6   = sim.getObject("/target[6]") -- Aliado6 [-5  5]
       targetHandle7   = sim.getObject("/target[7]") -- Aliado7 [ 0  5]
       targetHandle8   = sim.getObject("/target[8]") -- Aliado8 [ 5  5]

       baseHandle0      = sim.getObject(":/Quadcopter[0]")
       baseHandle1      = sim.getObject(":/Quadcopter[1]")
       baseHandle2      = sim.getObject(":/Quadcopter[2]")
       baseHandle3      = sim.getObject(":/Quadcopter[3]")
       baseHandle4      = sim.getObject(":/Quadcopter[4]")
       baseHandle5      = sim.getObject(":/Quadcopter[5]")
       baseHandle6      = sim.getObject(":/Quadcopter[6]")
       baseHandle7      = sim.getObject(":/Quadcopter[7]")
       baseHandle8      = sim.getObject(":/Quadcopter[8]")

    if targetHandle0   == -1 and
       targetHandle1   == -1 and
       targetHandle2   == -1 and
       targetHandle3   == -1 and
       targetHandle4   == -1 and
       targetHandle5   == -1 and
       targetHandle6   == -1 and
       targetHandle7   == -1 and
       targetHandle8   == -1 then

        sim.addStatusbarMessage('Error al obtener el handle del target')
    else
        sim.addStatusbarMessage('Handle de los targets obtenido correctamente')
    end
end

function sysCall_actuation()

end

function sysCall_sensing()
    -- put your sensing code here
end

function sysCall_cleanup()
    -- do some clean-up here
end
```